A FRAMEWORK FOR A DECENTRALIZED AND INTELLIGENT INTRUSION DETECTION SYSTEM IN IOT ENVIRONMENTS

SUBMITTED BY

SAWIRA QAMAR

ROLL NO: 3012-241003

Department of Computer Science & Information Technology

HAZARA UNIVERSITY MANSEHRA

2026

A FRAMEWORK FOR A DECENTRALIZED AND INTELLIGENT INTRUSION DETECTION SYSTEM IN IOT ENVIRONMENTS

SUBMITTED BY: **SAWIRA QAMAR**
Department of Computer Science & Information Technology
Hazara University Mansehra

SUPERVISOR: **Dr. NOOR UL AMIN**
Associate Professor, Department of Computer Science & Information Technology
Hazara University Mansehra

Department of Computer Science & Information Technology

HAZARA UNIVERSITY MANSEHRA

2026

DECLARATION

I declare that this thesis entitled "**A Framework for a Decentralized and Intelligent Intrusion Detection System in IoT Environments**" is my original work carried out under the supervision of **Dr. Noor Ul Amin** in the **Department of Computer Science & Information Technology, Hazara University, Mansehra**, in partial fulfilment of the requirements for the award of the degree of **Master of Science in Computer Science**.

I further declare that this thesis has not been submitted, either in whole or in part, to any other university or institution for the award of any degree. Wherever the work of others has been used, it has been properly acknowledged through appropriate citations and references. I am solely responsible for the originality, authenticity, and accuracy of the work presented in this thesis.

SAWIRA QAMAR
MS Scholar

Dr. Noor Ul Amin
Associate Professor
Department of Computer Science &
Information Technology
Hazara University Mansehra

CERTIFICATE

It is certified that the thesis entitled **"A FRAMEWORK FOR A DECENTRALIZED AND INTELLIGENT INTRUSION DETECTION SYSTEM IN IOT ENVIRONMENTS"** submitted by **Sawira Qamar** for the award of Master of Science in Computer Science is based on the result of studies carried out by her in Hazara University, Mansehra. The dissertation or any part of it has not been previously submitted for any other degree.

Internal Examiner/Supervisor: _____

Internal Examiner: _____

External Examiner: _____

Chairman/HOD of Department: _____

Dr. Mushtaq Ali
Department of Computer Science &
Information Technology
Hazara University, Mansehra

ACKNOWLEDGMENTS

All praise is to **Almighty Allah (SWT)** for His countless blessings, mercy, and guidance. Peace and blessings be upon **Prophet Muhammad (ﷺ)**, whose teachings are a source of guidance for all humanity.

I would like to express my sincere gratitude to my respected supervisor, **Dr. Noor Ul Amin**, for his valuable guidance, continuous encouragement, insightful suggestions, and unwavering support throughout this research. His knowledge, patience, and valuable supervision greatly enriched this research.

I am also thankful to all the faculty members of the Department of Computer Science & Information Technology, Hazara University, Mansehra, for their guidance, cooperation, and for providing an excellent academic environment during my studies.

I express my heartfelt gratitude to my beloved parents for their unconditional love, endless prayers, sacrifices, and constant encouragement. Their support has always been my greatest source of strength and inspiration.

SAWIRA QAMAR

LIST OF ABBREVIATIONS

API	Application Programming Interface
CSV	Comma-Separated Values
DDoS	Distributed Denial of Service
EDA	Exploratory Data Analysis
EVM	Ethereum Virtual Machine
GNB	Gaussian Naive Bayes
H-IoT	Healthcare Internet of Things
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IPFS	InterPlanetary File System
KNN	K-Nearest Neighbor
LSTM-AE	Long Short-Term Memory Auto-Encoder
LSVM	Linear Support Vector Machine
MAC	Medium Access Control
MLP	Multilayer Perceptron
MQTT	MQ Telemetry Transport
PBFT	Practical Byzantine Fault Tolerance
RFID	Radio-Frequency Identification
RIP	Routing Information Protocol

CONTENTS

A FRAMEWORK FOR A DECENTRALIZED AND INTELLIGENT INTRUSION DETECTION SYSTEM IN IOT ENVIRONMENTS i

DECLARATION iii

CERTIFICATE iv

ACKNOWLEDGMENTS v

LIST OF ABBREVIATIONS vi

CONTENTS vii

List of Figures xi

ABSTRACT xii

CHAPTER 1 INTRODUCTION 1

1.1 Background of the Study 2

1.2 Research Motivation 3

1.3 Problem Statement 3

1.4 Research Gap 4

1.5 Research Objectives 4

1.6 Research Questions and Hypotheses 5

1.7 Research Questions 5

1.8 Scope of the Study 5

1.9 Research Limitations 6

1.10 Thesis Organization 6

CHAPTER 2 LITERATURE REVIEW 7

2.1 Introduction 8

2.2 Current Trends and Future Prospects of IoT 8

2.3 Smart Cities 8
2.4 Internet of Things (IoT) Security Issues 9
2.5 Edge Computing and Software-Defined Networking 10
2.6 Blockchain-Enabled Internet of Things (IoT) 10
2.7 Blockchain and Machine Learning for Malware Detection and Mitigation 11
2.7.1 Malware Detection 11
2.7.2 Malware Mitigation 12
2.8 Datasets 14
2.8.1 CICIoT2023 Dataset 14
2.8.2 EdgeIIoT Dataset 14
2.9 Summary and Conclusion 15
CHAPTER 3 METHODOLOGY 18
3.1 Introduction 19
3.2 Proposed Blockchain-Based Smart City IoT Network 19
3.3 Machine Learning Model Evaluation Pipeline 20
3.4 Dataset Preparation 21
3.4.1 CICIoT2023 Dataset 22
3.4.2 Dataset – EdgeIIoT 22
3.5 Data Preprocessing 23
3.6 Feature Engineering 24
3.6.1 Feature Selection 24
3.6.2 Feature Scaling 25
3.6.3 Hyperparameter Tuning 25
3.6.4 Categorical Encoding 26
3.7 Machine Learning Models 26

3.7.1 Gaussian Naive Bayes (GNB) 27
3.7.2 Decision Tree (DT) 27
3.7.3 Random Forest (RF) 28
3.7.4 Linear Support Vector Machine (LSVM) 28
3.7.5 K-Nearest Neighbors (KNN) 28
3.7.6 XGBoost 29
3.7.7 Multi-Layer Perceptron (MLP) 29
3.8 Model Training and Testing 30
3.9 Model Evaluation Metrics 31
3.10 Blockchain-Based Prototype Development 32
3.11 Cooja-Based IoT Network Simulation 33
3.12 Wireshark Traffic Capture and Analysis 33
3.13 Blockchain and Smart Contract Implementation 34
3.14 Automated Attack Mitigation 34
3.15 Overall Proposed Framework 35
3.16 Research Methodology Summary 36
CHAPTER 4 RESULTS AND DISCUSSION 37
4.1 Introduction 38
4.2 Experimental Setup 38
4.3 Exploratory Data Analysis 39
4.3.1 CICIoT2023 Dataset 39
4.3.2 EdgeIIoT Dataset 44
4.4 PCAP to CSV Data Conversion 46
4.5 Data Preprocessing and Feature Selection Results 46
4.5.1 Data Cleaning 47

4.5.2 Data Splitting and Class Balancing 47
4.6 Feature Engineering 48
4.6.1 Feature Selection 48
4.6.2 Hyperparameter Tuning 53
4.7 Machine Learning Model Evaluation 54
4.7.1 Binary Classification Measurement Metrics 54
4.7.2 Research Question 1 and Hypothesis 1 56
4.7.3 Research Question 2 and Hypothesis 2 57
4.7.4 Research Question 3 and Hypothesis 3 57
4.8 Multiclass Classification 62
4.8.1 Multiclass Classification Measurement Metrics.....63
4.9 Summary 73
CHAPTER 5 CONCLUSION AND FUTURE WORK.....75
5.1 Conclusion.....76
5.2 Conclusions 77
5.3 Contributions to Body of Knowledge.....78
5.4 Recommendations for Future Research 79
5.4.1 Evaluation Using Additional IoT Datasets: 79
5.4.2 Lightweight Machine Learning Models: 79
5.4.3 Large-Scale IoT Testbed Evaluation: 79
5.4.4 Blockchain Interoperability: 79
5.4.5 Adaptive and Online Learning: 80
REFERENCES 81

List of Figures

Figure 3.1 Proposed Blockchain-based Smart City IoT Network Diagram.....20
Figure 3.2 Machine Learning Model Evaluation of Intrusion Detection.....21
Figure 3.3 EdgeIIoT HTTP Flood Packet in Wireshark.....24
Figure 3.4 Flowchart for simulation of the Prototype.....31
Figure 4.1 CICIoT Attack Vs Benign Distribution.....40
Figure 4.2 CICIoT Protocols Vs Attack Distribution.....43
Figure 4.3 EdgeIIoT Attack Vs Benign Distribution.....44
Figure 4.4 CICIoT Multiclass Attack Category Allocation Using SMOTE Sampling.....48
Figure 4.5 Pearson Correlation Coefficient Attack Flag (CICIoT).....49
Figure 4.6 Correlation Coefficient of Selected 14 Features.....52
Figure 4.7 Correlation Coefficient of Selected 14 Features.....53
Figure 4.8 Binary Classification Accuracy Standard Deviation.....56
Figure 4.9 Binary Classification Average of Accuracy.....57
Figure 4.10 Binary Classification False Positive Rate.....61
Figure 4.11 Multiclass Classification Accuracy Standard Deviation.....65
Figure 4.12 Multiclass Classification Average of Micro Accuracy.....66
Figure 4.13 Multiclass Classification False Positive Rate.....69
Figure 4.14 Blockchain Enabled Machine Learning IoT Framework in UML.....70
Figure 4.15 Smart Contracts Deployed in the Ethereum Virtual Machine.....72
Figure 4.16 Register New Devices in the Ethereum Virtual Machine.....72
Figure 4.17 Remove Compromised Device in the Ethereum Virtual Machine.....73

ABSTRACT

Internet of Things (IoT) is revolutionizing the modern world by bringing billions of devices together across all industries, homes and cities. This link can enable automation and data sharing, but also exposes them to cyber risks. Given these challenges, the study proposes the machine learning and blockchain based decentralized and intelligent intrusion detection system. The framework is made up of seven algorithms: Random Forest, Decision Tree, Linear SVM, Gaussian Naive Bayes, KNN, XGBoost, and MLP, which are compared and tested to detect anomalies within IoT networks. Among the evaluated machine learning models, XGBoost achieved strong classification performance, while Gaussian Naive Bayes (GNB) demonstrated the shortest processing time. These findings highlight the importance of considering both classification performance and computational efficiency when selecting a model for IoT intrusion detection.

Smart contracts are used for threat response and to reliably maintain devices. The proposed system will target to further enhance the security of the IoT system using the ideas of predictive analysis and immutability of records. It provides a scalable solution which is flexible enough to fit various IoT architectures. The study helps in building secure and transparent IoT ecosystems that are resilient. This integration of machine learning intelligence and blockchain decentralization improves intrusion detection, mitigates risks of malware and provides a secure operation for IoT systems. The study offers a contribution to both practitioners and researchers that are interested in enhancing the IoT cybersecurity frameworks.

CHAPTER 1
INTRODUCTION

1.1 Background of the Study

The Internet of Things (IoT) is a technology that enables physical devices, sensors, and systems to be connected through communication networks, allowing data to be exchanged, monitored, and automated [3]. Smart home, smart healthcare, smart industries, smart transportation and smart cities are the fields where IoT application is gaining momentum. As connected devices proliferate, so does the efficiency and functionality that has come with them, but it has also raised the risks for IoT networks of cybersecurity attacks [5].

IoT devices are deployed in distributed and heterogeneous environments and many devices are resource constrained with limited computational and security resources [1]. All these properties make IoT networks susceptible to Distributed Denial-of-Service (DDoS) attacks, malware, botnets, unauthorized access, and other malicious activities. Thus, it is necessary to have effective intrusion detection mechanisms to detect abnormal network traffic and secure IoT environments [3].

Intrusion detection can be aided by Machine Learning (ML) which can learn patterns of network traffic and differentiate between benign and malicious traffic. But such a centralized security strategy can raise issues of trust, data integrity and centralized control. Decentralization, transparency, and tamper resistance are the benefits blockchain technology can offer to manage data, complementing machine learning-based intrusion detection [4]. Thus, the incorporation of Machine Learning and Blockchain can serve as a basis to achieve a more intelligent, decentralized, and secure IoT intrusion detection system [6].

As the number of connected IoT devices continues to grow, security mechanisms are needed that can accurately identify attacks and effectively respond to them. Older intrusion detection methods might not be enough to detect evolving attack patterns of today's IoT devices. Machine Learning can be used to analyze network traffic and detect patterns that are related to malicious behavior[12].

The motivation for this research is to compare and contrast the various machine learning models to determine which model offers a good intrusion detection capability in IoT environments. The research also seeks to recognize significant network traffic aspects which can help in accurate classification of benign and malicious traffic [7].

But when an attack is detected, intrusion detection can be an insufficient method. Automated mitigation should also be supported by a security framework and trust should be maintained in the IoT environment. The decentralized nature and smart contracts of blockchain technology can offer a solution to handle security-related activities and information integrity [14]. In this

context, this research explores the possibility of combining the Machine Learning and Blockchain technologies to propose an end-to-end solution for intelligent intrusion detection and automated attack prevention in IoT networks [14][15].

1.2 Research Motivation

As the number of connected IoT devices continues to grow, security mechanisms are needed that can accurately identify attacks and effectively respond to them. Older intrusion detection methods might not be enough to detect evolving attack patterns of today's IoT devices. Machine learning can be used to analyze network traffic and detect patterns that are related to malicious behavior [8].

The motivation for this research is to compare and contrast the various machine learning models to determine which model offers a good intrusion detection capability in IoT environments. The research also seeks to recognize significant network traffic aspects, which can help in accurate classification of benign and malicious traffic [9].

But when an attack is detected, intrusion detection can be an insufficient method. Automated mitigation should also be supported by a security framework, and trust should be maintained in the IoT environment. The decentralized nature and smart contracts of blockchain technology can offer a solution to handle security-related activities and information integrity. In this context, this research explores the possibility of combining the machine learning and blockchain technologies to propose an end-to-end solution for intelligent intrusion detection and automated attack prevention in IoT networks [10].

1.3 Problem Statement

The rise of Internet of Things (IoT) devices has brought with it the threat of cyberattacks on IoT. There are various forms of malicious activities that can happen in IoT networks, such as malware attacks, Distributed Denial-of-Service (DDoS) attacks, and botnet activities. These attacks can impact the availability, reliability, and security of IoT services.

One of the difficulties is the correct detection of malicious network traffic and keeping the false positive rate low. The performance of detection varies among different machine learning models, and the selection of appropriate network traffic features is crucial to make effective classification. Moreover, merely identifying an attack is not enough if the system is not able to react to the identified attack in a timely and automated fashion.

A framework to identify critical network traffic characteristics, assess the suitability of various machine learning models for intrusion detection, and facilitate automatic attack mitigation is

therefore required. By combining machine learning with blockchain, one could potentially have an intelligent system to detect, manage security, and automate responses in IoT systems. This research aims to fill this gap by creating a prototype combining machine learning (ML) and blockchain to detect intrusions and automatically mitigate attacks.

1.4 Research Gap

Existing Research	Research Gap
Uses one or two ML algorithms	No comprehensive comparison of multiple ML models
Focuses mainly on intrusion detection	Limited automated attack mitigation
Uses Blockchain for trust management	Limited integration with intelligent IDS
Mostly theoretical frameworks	Lack of a complete end-to-end prototype framework

1.5 Research Objectives

The main objectives of the study is to test the new model that would unify blockchain technology and machine learning to improve the process of IoT attack detection and mitigation.

i. To analyze and test the performance of the seven ML algorithms for IoT intrusion detection.
ii. To use feature selection techniques for extracting the relevant features in IoT malware detection.
iii. To develop an end-to-end framework integrating Blockchain with Machine Learning for automated mitigation and intrusion detection.
iv. To evaluate the proposed framework using Accuracy, F1-score, False Positive Rate, and Response Time.

Identify the Critical Contributors Among Independent Variables:
For effective identification of intrusions in IoT traffic, it is important to have knowledge of the critical variables out of 3 independent variables. In this step, feature selection and feature analysis are done to select the variables that are important in distinguishing the malicious activities. These key contributors will be identified using Pearson correlation, variance threshold, mutual information, and chi-square feature selection methods. The machine learning models can be fine-tuned to make them more effective by concentrating on the most influential factors. Such a narrow approach enables the creation of more accurate models and subsequently faster and more efficient models that are more suitable for smart city applications in real time.

The ability to merge machine learning with blockchain technology is an attractive solution for IoT environments with respect to mitigating cyberattacks. Blockchain's decentralization and immutability guarantee data integrity and trust, and machine learning algorithms add an extra degree of safety to protect against threats. This research will be used to illustrate the proof-of-concept implementation of this integrated approach to demonstrate the effectiveness of this approach. Using machine learning algorithms to analyze IoT traffic and detect any anomalies, and blockchain technology to secure and ensure that all transactions are secure and accurate. A cyberattack will be developed to assess the effectiveness of the system. The outcomes will emphasize the ability of ML and blockchain to offer a robust solution to withstand cyber threats.

1.6 Research Questions and Hypotheses

Research hypotheses are targeted towards testing the seven machine learning predictive models and determining which model is able to best detect malware attacks on an IoT network. Other than this, the objective of this research is to recommend a suitable system that can decrease the risk of attacks from IoT malware through the smart contract on the Ethereum blockchain.

1.7 Research Questions

In this study, the following research questions will be addressed:

i. Which network traffic features are most effective for classifying the benign traffic from IoT malware attacks?
ii. Which machine learning model demonstrates the highest detection accuracy and overall performance while maintaining low false positive rates?
iii. How can an end-to-end prototype integrating Machine Learning and Blockchain support intrusion detection and automated attack mitigation in IoT environments?

1.8 Scope of the Study

This research focuses on the development and evaluation of an intelligent and decentralized intrusion detection framework for IoT environments. The study evaluates seven machine learning models: Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), eXtreme Gradient Boosting (XGBoost), and Multi-Layer Perceptron (MLP).

Two public IoT datasets, CICIoT and EdgeIIoT, are used to evaluate the machine learning models for binary and multiclass intrusion detection. The study includes data preprocessing, feature selection, feature scaling, model training, and performance evaluation using accuracy, precision, recall, F1-score, response time, and false positive rate.

The research also covers the development of an end-to-end prototype that integrates the selected machine learning model with a permissioned Ethereum blockchain and smart contracts. The prototype demonstrates intrusion detection and automated mitigation by identifying and removing compromised IoT devices from the blockchain network. The blockchain prototype is implemented in a local Ethereum Virtual Machine (EVM) development environment.

1.9 Research Limitations

This research has several limitations. First, the study is limited to the CICIoT and EdgeIIoT datasets, which may not represent all possible IoT network environments and attack scenarios. Second, the evaluation is limited to seven machine learning models: GNB, DT, RF, LSTM, KNN, XGBoost, and MLP. Third, the blockchain-based prototype is implemented in a local Ethereum Virtual Machine (EVM) development environment rather than a large-scale real-world IoT network. Therefore, the performance of the proposed framework may vary when deployed in different real-world environments, with different devices, network conditions, and attack patterns.

1.10 Thesis Organization

This thesis is organized into five chapters.

Chapter 1 presents the introduction, research motivation, problem statement, thesis statement, research objectives, research questions and hypotheses, scope, and limitations of the research. Chapter 2 presents the literature review related to IoT security, intrusion detection, machine learning, and blockchain technologies. Chapter 3 describes the research methodology, including dataset preparation, feature selection, machine learning models, performance evaluation, and the development of the proposed prototype. Chapter 4 presents the experimental results and discusses the performance of the evaluated machine learning models and the proposed blockchain-based security framework. Chapter 5 concludes the research, summarizes the main findings, discusses the contributions of the study, and presents recommendations for future research.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

This chapter presents a review of the literature related to Internet of Things (IoT) security, intrusion detection, Machine Learning (ML), and Blockchain technology. It discusses the development and applications of IoT and the security challenges associated with the increasing number of connected devices [15]. The chapter also reviews major IoT security threats and existing approaches for detecting and mitigating malicious activities [16].

Furthermore, the chapter discusses Machine Learning techniques used for IoT intrusion detection and examines the role of Blockchain technology in providing decentralized and secure security management. The integration of Machine Learning and Blockchain for IoT security is also reviewed to identify existing approaches and their limitations. The CICIoT and EdgeIIoT datasets used in this research are discussed, followed by a review of the research gap that motivates the proposed framework [18].

2.2 Current Trends and Future Prospects of IoT

The Internet of Things (IoT) has developed rapidly with the increasing availability of connected devices, sensors, communication technologies, and intelligent computing systems. IoT enables physical devices to collect, exchange, and process data through interconnected networks. Its applications have expanded across smart homes, healthcare, industrial systems, transportation, agriculture, and smart cities [17].

Recent developments in IoT have focused on improving connectivity, automation, intelligence, and real-time data processing. Technologies such as edge computing, cloud computing, Artificial Intelligence (AI), and Machine Learning (ML) are increasingly used to process IoT data and support intelligent decision-making [19]. These developments have improved the efficiency and functionality of IoT systems but have also increased the complexity of securing interconnected devices and networks [25].

The future development of IoT is expected to involve a greater number of connected devices and increased use of intelligent and distributed technologies. As IoT environments continue to expand, effective security mechanisms will be required to protect devices, network communication, and sensitive data from cyber attacks. Therefore, secure and intelligent approaches to IoT intrusion detection remain an important research area [21].

2.3 Smart Cities

Smart cities use Internet of Things (IoT) technologies to connect devices, sensors, communication networks, and services for improving the management of urban environments [22]. IoT-enabled systems can support applications such as intelligent transportation, energy management, environmental monitoring, public safety, healthcare, and infrastructure management [20].

The increasing deployment of connected devices in smart cities enables continuous collection and exchange of large amounts of data. This connectivity can improve the efficiency and quality of urban services, but it also increases the potential attack surface of city networks. Compromised IoT devices can be exploited to perform malicious activities and attacks that may affect the availability, integrity, and reliability of smart-city services [23].

Therefore, security is an important requirement for smart-city IoT environments. Effective intrusion detection mechanisms are needed to identify abnormal network traffic and malicious activities [24]. Machine Learning can support the detection of complex attack patterns, while Blockchain can provide decentralized and tamper-resistant management of security-related information. These technologies can contribute to improving the security and trustworthiness of IoT-based smart-city environments [28].

2.4 Internet of Things (IoT) Security Issues

The rapid growth of IoT has increased the number of connected devices and expanded the potential attack surface of IoT networks. IoT devices can be vulnerable because they are deployed in heterogeneous and distributed environments, and many devices have limited computational and security resources. These characteristics can make IoT systems attractive targets for cyberattacks [25].

Common threats against IoT environments include malware, botnet attacks, Distributed Denial-of-Service (DDoS) attacks, unauthorized access [27], data manipulation, and other malicious activities. Malware-infected IoT devices can be controlled by attackers and used as part of botnets to launch attacks against other systems. DDoS attacks can generate large volumes of malicious traffic and affect the availability of IoT services [26].

IoT security is also affected by weaknesses in device authentication, communication protocols, software configurations, and access-control mechanisms. The large number of connected devices makes it difficult to monitor every device continuously and identify malicious behavior using conventional security mechanisms alone [30] [21].

Therefore, effective intrusion detection is an important requirement for protecting IoT environments. Machine Learning can analyze network traffic and learn patterns associated with

benign and malicious activities, while Blockchain can support decentralized and tamper-resistant security management [29]. The combination of these technologies provides a potential approach for improving intrusion detection and supporting automated mitigation of attacks in IoT environments [34].

2.5 Edge Computing and Software-Defined Networking

Edge Computing is an approach in which data processing and computing resources are placed closer to IoT devices and data sources [31]. By processing data near the edge of the network, it can reduce communication delays and support faster responses for IoT applications. This characteristic is particularly useful for IoT environments where real-time monitoring and security responses are required [35].

Software-Defined Networking (SDN) separates the network control plane from the data plane and provides centralized management of network resources. In IoT environments, SDN can support flexible network management, traffic monitoring, and security-policy enforcement. It can also help security systems identify and control suspicious network traffic [32].

The combination of Edge Computing and SDN can support efficient management and security of large-scale IoT networks [33]. However, the distributed nature and increasing complexity of IoT environments still create challenges for detecting and responding to sophisticated attacks. Therefore, intelligent intrusion detection mechanisms are required to identify malicious traffic and support timely security responses [43].

2.6 Blockchain-Enabled Internet of Things (IoT)

Blockchain is a distributed ledger technology that enables information to be recorded and shared among multiple participants without relying on a single centralized authority [36]. Transactions or records stored on a blockchain are organized into blocks and linked together, which helps provide transparency and resistance to unauthorized modification [44].

Blockchain technology can provide useful security properties for IoT environments, including decentralization, data integrity, transparency, and trust among participating devices [38]. A permissioned blockchain can further restrict participation to authorized entities, making it suitable for controlled IoT environments where security and access management are important.

Smart contracts are programs deployed on a blockchain that can automatically execute predefined actions when specified conditions are satisfied [37]. In IoT security, smart contracts can be used to manage device registration, access control, security-related information, and automated responses to detected threats [40].

Despite these advantages, blockchain-based IoT security can face challenges related to scalability, computational requirements, latency, and integration with resource-constrained IoT devices. Therefore, blockchain should be carefully integrated with other security technologies to provide an effective solution for IoT environments [41].

2.7 Blockchain and Machine Learning for Malware Detection and Mitigation

Machine Learning (ML) and Blockchain are two technologies that can complement each other in improving the security of IoT environments [51]. Machine Learning can analyze network traffic and identify patterns associated with benign and malicious activities, while Blockchain can provide decentralized and tamper-resistant management of security-related information [42].

Machine Learning-based intrusion detection systems can be trained using network traffic data to classify normal and malicious activities [36]. Different ML algorithms may provide different levels of detection accuracy, precision, recall, F1-score, response time, and false positive rate [44]. Therefore, evaluating multiple machine learning models and identifying important network traffic features are important for developing an effective IoT intrusion detection system [43].

Blockchain can complement ML-based intrusion detection by providing a decentralized mechanism for managing security information and trusted interactions among participating devices. Smart contracts can be used to automate predefined security actions when malicious activity is detected. This can support automated mitigation and reduce dependence on a centralized security authority [45].

The integration of Machine Learning and Blockchain can therefore provide an end-to-end approach in which ML is used for intelligent intrusion detection and Blockchain is used for decentralized security management and automated response. However, challenges related to computational overhead, scalability, response time, and integration with resource-constrained IoT devices need to be considered [52].

Based on these considerations, this research investigates the integration of Machine Learning and Blockchain to develop a prototype for intrusion detection and automated attack mitigation in IoT environments.

2.7.1 Malware Detection

[22] Introduce a healthcare IoT network machine learning system with blockchain. The proposed three-layered architecture includes a user data layer, a blockchain layer with a machine learning analytical engine, and a cloud server layer to store all of the information in one place. Support Vector Machine (SVM) is used for malware detection, data classification, and traffic monitoring. The system has been able to detect intrusions but has problems with performance and scalability. The model is only for machine learning applications with the whole network centralized on a cloud environment, which makes it vulnerable to single-point failure vulnerabilities [22.

[7] propose an Ethereum-based solution and an "ensemble-stacked" [7] machine-learning model. The paper relies on smart contracts to analyze the data, the InterPlanetary File System (IPFS) to store the data, and an ensemble machine learning model to detect the data. The suggested system verifies the information analysts and removes network nodes if any intrusion is identified. Intrusion detection would be identified by the machine learning module, and the information would be updated by the IPFS through the whole blockchain IoT network [7].

[42] presented a new Privacy-Preserving and Secure Framework (PPSF) for smart cities, which will be based on an InterPlanetary File System (IPFS) system using blockchain technology and a machine learning model called Gradient Boosting Anomaly Detector (GBAD). From the experimental results obtained, they have demonstrated that their proposed blockchain IoT architecture can be implemented and is beneficial in comparison to the current centralized server-based architecture, particularly in terms of information sharing between all the involved parties.

To detect DDoS attacks, [35] propose the use of a blockchain-based IoT environment, along with the Harris Hawks optimization algorithm and a sine cosine algorithm for feature selection, and a long short term memory (LSTM) autoencoder model for intrusion detection. Their study states that they have positive detection results, but they do not mention the application of blockchain in mitigation.

2.7.2 Malware Mitigation

The strategies to limit attacks are primarily focused on blockchain and smart contracts to disseminate information about attacks, such as whitelisting/blacklisting of IP addresses. One of the whitelisting alternatives is to establish an identifier for each device and give it to the network. After changing the content identifier of a device, it will no longer be able to communicate with other devices [15]. An alternative method is to provide an authentication and verification process on the application level. Thereafter, malicious device IP addresses are

tracked and recorded in the blockchain, keeping them from connecting and conversing with different gadgets [26].

The models of arbitrary physically unclonable functions (PUFs) are suggested as an authenticating mechanism and added in a permissioned blockchain to resist DDoS attacks, as mentioned in the [6] paper. If an attack is detected, the system will send out all intrusion messages to all nodes of the blockchain and append the infected IP addresses [6] to the blacklist.

An idea that has been proposed is to couple Ethereum smart contracts with routing information protocol (RIP). These smart contracts are supposed to maintain the routing tables of the RIP and to identify suspicious traffic patterns such as large volumes of traffic or gas consumption. The smart contracts can be used to drop packets or reroute traffic during attacks as described in [11].

[19] use an Ethereum consortium network-based blockchain and a deep learning network with LSTM to identify and block the attacks. Attack detection is achieved using LSTM, and blacklist/whitelist communication is achieved by smart contracts, also integrated into the architecture. The architecture's LSTM detects attacks, and the communication between the black and whitelisted IP addresses is managed by the smart contract that is integrated into the architecture. In most cases, the same deep learning model can be used to block an attack in milliseconds, the authors say, and the model can detect attacks in 10-60 seconds. In this work, [57] propose a three-policy framework for access, communication, and usage of blockchain smart contracts and embed botnet detection and mitigation tools into the policies. Suspicion of intrusion is raised if there is any breach of policy. When a detection is triggered, the mitigation steps to follow include, for example, credential updates, IP and domain blocking, or rebooting the devices [57].

For this, Jiang et al. have used Ethereum smart contracts to collect and broadcast live data and adopted Long-Short-Term Memory (LSTM) for the detection of atypical patterns.

The suspicious source is assumed by comparing the data with the values of standard deviations. Once the source is marked, the blockchain notifies all the nodes to update their routing or to drop the packet [31].

The combination of blockchain and SDN is suggested to enhance the SDN central control system's security. To this end, [32] suggest the following: A secure middle layer consisting of blockchain, which is placed between the control layer and the data layer. This layer tracks newly established network flows and generates lists of possible threats that are then stored on

the blockchain, which are tamper-proof and shared. Also, in this secure layer, smart contracts can ring an alarm that can remove the harmful packets that are sent out to the network switches.

2.8 Datasets

This research uses publicly available IoT network traffic datasets to evaluate machine learning models for intrusion detection. The datasets provide network traffic samples representing benign and malicious activities and are used to train and evaluate the machine learning models.

2.8.1 CICIoT2023 Dataset

The CICIoT2023 dataset is an IoT network traffic dataset developed for evaluating cybersecurity and intrusion detection approaches in IoT environments. It contains benign traffic as well as traffic associated with different types of IoT attacks. In this research, the dataset is used for binary and multiclass classification experiments and for evaluating the performance of the selected machine learning models.

2.8.2 EdgeIIoT Dataset

The EdgeIIoT dataset is another publicly available dataset containing network traffic from an IoT and Industrial IoT environment. It includes benign and malicious traffic generated from different attack scenarios. In this research, the dataset is used to evaluate the generalization and detection performance of the selected machine learning models under different IoT traffic conditions.

The use of two different datasets allows the machine learning models to be evaluated across different network traffic characteristics and attack scenarios. The datasets are subjected to preprocessing and feature-selection techniques before being used for model training and evaluation.

Table 2-1 Public Datasets – BoT-IoT, CIC DDoS 2019, CICIoT, and EdgeIIoT

Dataset	Number of Records	Categories of Attacks	Number of IoT Devices	Number of Features	Date Published
BoT-IoT	~72 million	5	*5*	*32*	*Nov 2018*
CIC DDoS 2019	~32 million	*12*	*4*	*80*	*Oct 2019*
EdgeIIoT	~18 million	*14*	*10*	*61*	*Apr 2022*

CICIoT	~46 million	*33*	*105*	*47*	*Jun 2023*

The comparison of the 4 datasets studied is presented in Table 2-1. For this research, we have used datasets with machine learning model evaluation for IoT malware detection, EdgeIIoT, and CIC-IoT, because they are recent (less than 2 years old) with a large number of devices, vast data records, and a large number of attack categories. The other motivation for selecting the CICIoT and EdgeIIoT datasets is their use of many different protocols. Many protocols are industrial standards, and they are therefore more susceptible to protocol-level attacks than are application-layer attacks, as they are not so rapidly moved to new versions as are application-layer protocols.

There are different types of protocols, which offer a more authentic data source for a machine learning assessment.

2.9 Summary and Conclusion

The IoT has brought about great technological progress in smart cities all over the world, improving our life quality. The Smart City Index (SCI) of the Smart City Observatory (SCO) at the International Institute for Management Development (IMD) World Competitiveness Center (WCC) is a list that ranks 141 cities by their effective application of technology to solve urban problems and elevate their quality of life [28]. However, the pervasive use of IoT also leaves it vulnerable to attacks due to differences in architectures, protocols, and topologies. One of the crucial aspects of IoT networks, especially in smart cities, is security, trust and privacy. These networks are susceptible to various kinds of cyberattacks particularly limited resource IoT networks which is highly susceptible to the advanced attacks on the security [73]. These weaknesses are exploited to get information, to launch ransomware attacks or to cause disruption in critical infrastructure, including during this geopolitical conflict [44 [45].

However, researchers have proposed several solutions to improve the security of IoT networks, such as the addition of cloud computing, edge computing, software-defined networking (SDN), and blockchain. The decentralization, transparency, consensus mechanisms, and cryptographic security features of blockchain technology offer a promising solution. Blockchain can help provide a secure and transparent environment, making sure that no single entity or point can be compromised.Blockchain can play a role in ensuring data integrity and eliminating any single points of failure for the IoT system [1].

In this research, two datasets of IoT devices—EdgeIIoT and CICIoT—are used to study defense mechanisms against the IoT malware attacks. The study is based on a lightweight Ethereum permissioned blockchain architecture and uses machine learning algorithms to assess the performance of seven machine learning models to see if they are a viable solution for resource-constrained IoT devices. The objective is to present an idea for a blockchain solution that can be implemented to prevent and react to attacks from the IoT. As seen in Table 2-2, we have done a comparison of our proposed solution with other studies in this field, and it was found that our solution has the following advantages.

Table 2-2 Comparison of the Proposed Solution Vs. Other Studies

AUTHOR (YEAR)	BLOCKCHAIN	DETECTION	MITIGATION	DATASET	RESEARCH LIMITATION
ESSAID ET AL. (2020)	Ethereum	LSTM	Blacklist and the whitelist IP addresses through the smart contracts.	None.	Machine learning detection takes 10-60 seconds, which is too long.
KUMAR ET AL. (2021)	Ethereum InterPlanetary File System	Gradient Boosting Anomaly Detector	None	ToN-IoT BoT-IoT	No mitigation was studied.
GAUR ET AL. (2022)	Not specified	SVM	None	None	No details on blockchain technology used and no mitigation was studied.
BABU ET AL. (2022)	Permissioned based blockchain with Arbiter Physically Unclonable Functions (PUFs)	Ensemble technique - Linear Regression, DT, RF, SVM.	Share the alerts to all the nodes in the blockchain and blacklist IP Address	CIC DDoS 2019	Focus only on DDoS detection. Detection speed is not measured.
JIANG ET AL. (2022)	Ethereum	Calculates the rate of packets and reports suspicious traffic for storing and sharing.	sharing the suspicious packets. Smart contract discard attacking packets sent to switches	None	No machine learning studied.

IBRAHIM ET AL. (2022)	Ethereum	Not specified	Blacklist the IP address of malicious devices	None	No machine learning studied.
CHENG ET AL. (2023)	Ethereum	Not specified	IPFS generates CID and uploads the CID to the blockchain for whitelisting	None	No machine learning studied.
KATIB & RAGAB (2023)	Not specified	LSTM	None	BoT-IoT	No detailed blockchain was specified.
MUSA BAIG ET AL. (2023)	Verifiable Byzantine Fault Tolerance (VBFT) blockchain	Ensemble Stacked Machine Learning	The malicious nodes are identified and removed	IoT botnet IoT fridge	Machine learning performance needs to be improved. Only focus on the DoS attack.
BHASKAR & PUROHIT. (2023)	Ethereum Smart contracts	Not specified	Smart contracts to control the RIP routing tables, drop packets entirely, or reroute traffic in the event of an attack.	None	No machine learning was studied.
JIANG ET AL. (2024)	Ethereum	LSTM	Reroute or drop the traffic sent by suspicious sources.	Real-time	Machine learning performance was not published.
JONY & ARNOB. (2024)	Not specified	LSTM	None	CICIoT	No details on blockchain. No mitigation was studied.
THIS PRAXIS (2025)	Ethereum Smart contracts	Seven machine learning models: GNB, DT, RF, LSVM, XGBoost, KNN, and MLP.	Smart contracts to remove infected devices from the blockchain	EdgeII0T CICIoT	Binary and multiclass classification. Machine learning performance metrics. A simulated end-to-end detection and mitigation prototype.

CHAPTER 3
METHODOLOGY

3.1 Introduction

This chapter presents the methodology used to develop and evaluate the proposed intelligent and decentralized intrusion detection framework for IoT environments. The methodology consists of two major phases: the comparative evaluation of seven machine learning models and the development of a blockchain-based prototype for intrusion detection and automated attack mitigation.

The first phase focuses on evaluating seven machine learning models: Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP). The CICIoT2023 and EdgeIIoT datasets are used for the experiments. The methodology includes data preprocessing, handling of class imbalance, feature selection, feature scaling, model training, and performance evaluation.

The second phase focuses on developing and simulating an end-to-end prototype that integrates the selected machine learning model with a permissioned Ethereum blockchain and smart contracts. Cooja is used for IoT network simulation, while Wireshark is used to capture network traffic. The detected malicious device information is transferred to the blockchain layer, where smart contracts support automated mitigation by removing or revoking access to compromised devices.

The machine learning models are evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). The methodology is designed to address the research questions and objectives of the study and to evaluate the effectiveness of integrating Machine Learning and Blockchain for IoT intrusion detection and automated attack mitigation.

3.2 Proposed Blockchain-Based Smart City IoT Network

The proposed smart city IoT network is organized into different domains, such as healthcare, energy, transportation, and smart homes. Each domain contains IoT devices, sensors, and actuators that communicate through their respective domain controllers.

The domain controllers are registered as authorized participants in the permissioned Ethereum blockchain network. They are responsible for managing IoT devices within their respective domains and communicating security-related information with the blockchain layer. Smart contracts are used to manage device registration and security actions within the network.

Trusted IoT devices can be registered through the smart contract, while unauthorized or compromised devices can be identified and removed from the trusted device registry. When an intrusion is detected, the malicious device information, such as its source IP address, can be forwarded to the smart contract. The smart contract can then execute the predefined mitigation function to revoke or remove the compromised device from the trusted network.

The proposed architecture focuses on the application layer of the blockchain and does not include a comparative analysis of different blockchain consensus mechanisms. The blockchain prototype is implemented in a simulated local environment to demonstrate the integration of intrusion detection and automated attack mitigation.

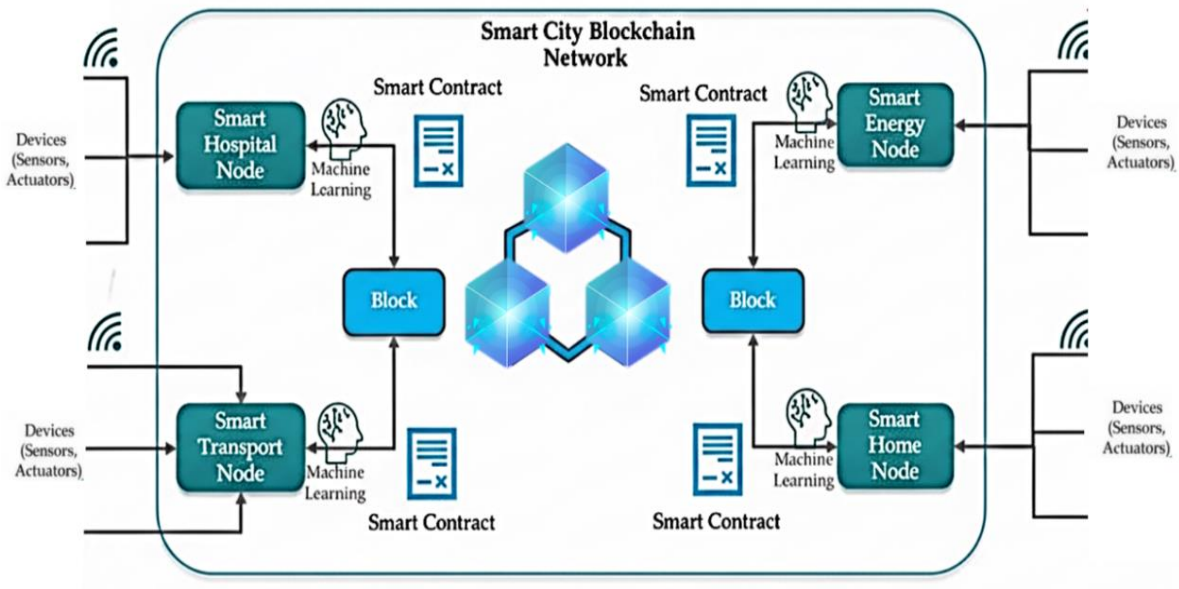


Figure 3.1 Proposed Blockchain-based Smart City IoT Network Diagram

3.3 Machine Learning Model Evaluation Pipeline

The machine learning model evaluation pipeline is designed to systematically evaluate seven machine learning models for IoT intrusion detection. The pipeline consists of dataset selection, data preprocessing, feature selection, feature scaling, model training, and performance evaluation.

The CICIoT2023 and EdgeIIoT datasets are used as the primary data sources. The datasets contain benign and malicious IoT network traffic and are prepared in a common format to support comparative evaluation. Data preprocessing is performed to clean the datasets and prepare them for machine learning analysis.

Feature selection is performed to identify the most relevant network traffic features and reduce redundant information. Four feature-selection techniques are applied: Pearson Correlation

Coefficient, Variance Threshold, Mutual Information, and Chi-Square Test. A combined feature-selection method is also considered by combining the results of these techniques. The selected feature sets are used to evaluate the effect of feature selection on model performance. Seven machine learning models—Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP)—are trained and evaluated. Both binary classification, which distinguishes benign and malicious traffic, and multiclass classification, which identifies different attack categories, are considered.

The datasets are divided into training and testing sets using stratified sampling to maintain the distribution of classes. The performance of the models is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). The results are compared to identify the most effective model and the most relevant network traffic features for IoT intrusion detection.

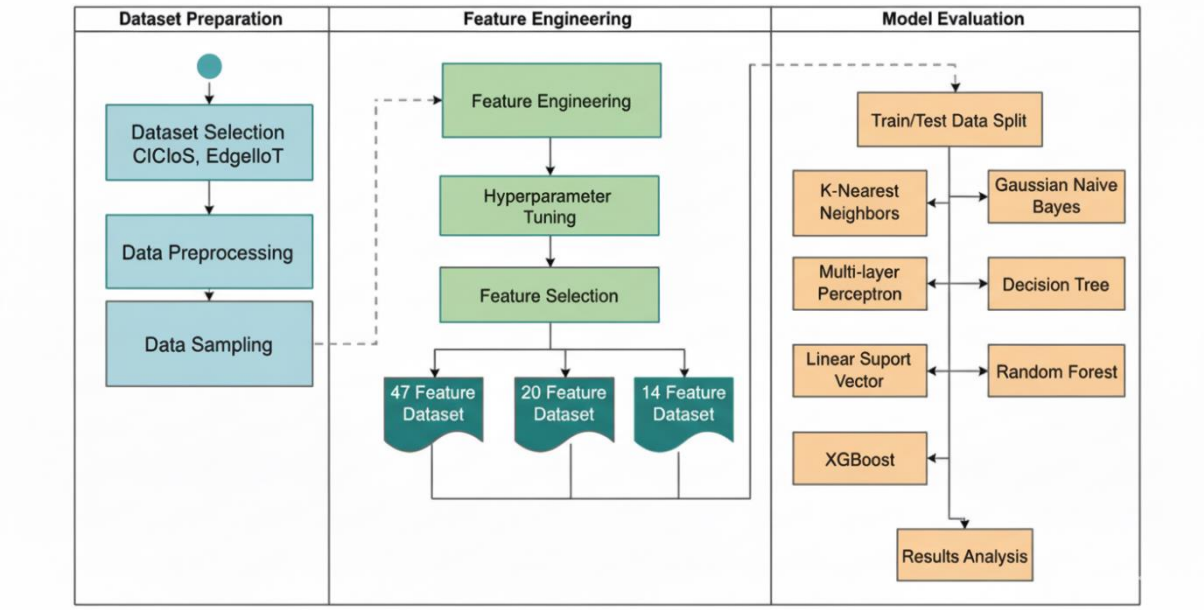


Figure 3.2 Machine Learning Model Evaluation of Intrusion Detection

3.4 Dataset Preparation

Two publicly available IoT network traffic datasets, CICIoT2023 and EdgeIIoT, are used in this research to evaluate the selected machine learning models for IoT intrusion detection. The datasets contain benign and malicious network traffic generated from different IoT environments and attack scenarios. The following sections describe the characteristics and preparation of each dataset for the machine learning experiments.

3.4.1 CICIoT2023 Dataset

The CICIoT2023 dataset was released by the Canadian Institute for Cybersecurity in 2023 and is designed to support cybersecurity research and intrusion detection in IoT environments. The dataset contains benign and malicious network traffic representing different categories of IoT attacks. In this research, CICIoT2023 is used to evaluate machine learning models for both binary and multiclass intrusion detection.

The dataset contains approximately 46 million records, 33 attack categories, 105 IoT devices, and 47 network traffic features. The attack categories include DDoS, DoS, Mirai, reconnaissance, web-based attacks, brute-force attacks, and spoofing. These characteristics make the dataset suitable for evaluating machine learning models under diverse IoT attack scenarios.

Before model training, the dataset is subjected to preprocessing and feature-selection procedures. The 47 available features are analyzed using Pearson Correlation, Variance Threshold, Mutual Information, Chi-Square Test, and a combined feature-selection approach. The selected feature sets are then used to evaluate the performance of the seven machine learning models.

3.4.2 Dataset – EdgeIIoT

The EdgeIIoT dataset is a publicly available dataset developed for cybersecurity research in Internet of Things (IoT) and Industrial Internet of Things (IIoT) environments. It contains network traffic representing both benign activities and different types of cyberattacks. In this research, the EdgeIIoT dataset is used to evaluate the performance of the selected machine learning models under different IoT network traffic conditions.

The dataset includes various attack categories, such as DDoS, DoS, reconnaissance, information gathering, injection, and other malicious activities. These diverse attack scenarios provide a suitable basis for evaluating machine learning models for intrusion detection.

Before model training, the EdgeIIoT dataset is subjected to data preprocessing and feature-selection procedures. The relevant network traffic features are identified using Pearson Correlation, Variance Threshold, Mutual Information, Chi-Square Test, and the Combined Method. The resulting feature sets are then used for binary and multiclass classification experiments with the seven selected machine learning models.

The use of EdgeIIoT together with CICIoT2023 enables the proposed approach to be evaluated across different IoT network environments and attack scenarios. This supports a more comprehensive assessment of the machine learning models for IoT intrusion detection.

3.5 Data Preprocessing

Data preprocessing is performed to prepare the CICIoT2023 and EdgeIIoT datasets for machine learning-based intrusion detection. The preprocessing stage ensures that the datasets are cleaned, transformed, and prepared in a suitable format for model training and evaluation. The preprocessing process includes handling missing values, removing irrelevant or redundant information, converting categorical information into a suitable numerical representation, and preparing the target labels for classification. The datasets are also examined to identify inconsistencies and ensure that the input data is suitable for the selected machine learning algorithms.

Since IoT datasets may contain imbalanced class distributions, the Synthetic Minority Over-sampling Technique (SMOTE) is applied to the training data to address class imbalance. SMOTE generates synthetic samples for minority classes and helps improve the ability of machine learning models to learn from underrepresented attack categories.

After preprocessing, the datasets are divided into training and testing sets using a 70:30 ratio. Stratified sampling is used to maintain the distribution of classes in both sets. Feature selection and feature scaling are then applied as required by the machine learning models. The prepared datasets are finally used for training and evaluating the seven selected machine learning models.

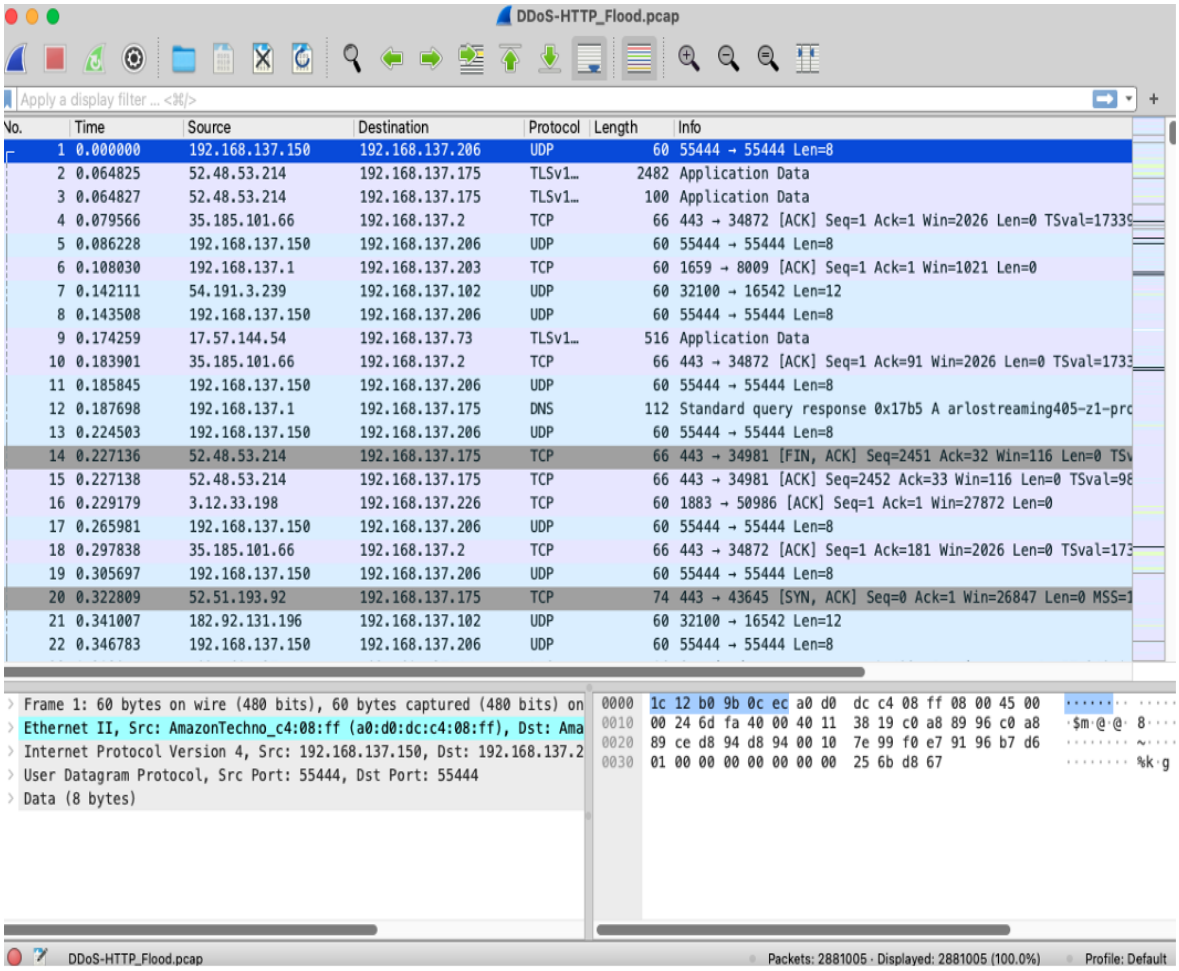


Figure 3.3 EdgeIIoT HTTP Flood Packet in Wireshark

3.6 Feature Engineering

Feature engineering is the process of preparing and transforming data into a suitable form for machine learning algorithms. In this research, feature engineering is performed to improve the efficiency and effectiveness of the machine learning models used for IoT intrusion detection. The feature engineering process consists of four main steps: feature selection, hyperparameter tuning, categorical encoding, and feature scaling.

3.6.1 Feature Selection

Feature selection is performed to identify the most informative network traffic features for IoT intrusion detection and to reduce redundant or less relevant information in the datasets. Selecting appropriate features can improve classification performance and reduce the computational requirements of machine learning models.

In this research, four feature-selection techniques are applied: Pearson Correlation Coefficient, Variance Threshold, Mutual Information, and Chi-Square Test. Each technique evaluates the

relevance of features from a different perspective. The selected features from these techniques are compared to determine their effect on the performance of the machine learning models.

In addition to the individual techniques, a Combined Method is used to integrate the feature-selection results obtained from the four techniques. This approach helps identify a common set of informative features that are consistently considered important across the different feature-selection methods.

The feature-selection process starts with the available network traffic features in the CICIoT2023 and EdgeIIoT datasets. The initial feature set is analyzed, and irrelevant or redundant features are reduced. The resulting feature sets are then used in the machine learning experiments to determine which features provide the most effective classification of benign and malicious IoT traffic.

The selected features are evaluated using the seven machine learning models: GNB, DT, RF, LSVM, KNN, XGBoost, and MLP**.** The performance of the models is compared using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR).

3.6.2 Feature Scaling

Feature scaling is performed to ensure that the selected network traffic features have an appropriate scale for machine learning algorithms. IoT network traffic features may have different numerical ranges, and large-scale features can influence the learning process of some machine learning models.

In this research, feature scaling is applied to the selected numerical features after the feature-selection stage. The scaling process transforms the feature values into a comparable range while preserving the relative information contained in the data.

The scaled features are then used for training and testing the selected machine learning models. Feature scaling is particularly important for models such as Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), and Multi-Layer Perceptron (MLP), which can be affected by differences in feature magnitude.

The same preprocessing and scaling procedure is applied consistently to the training and testing data to ensure a fair comparison among the seven machine learning models.

3.6.3 Hyperparameter Tuning

Hyperparameter tuning is performed to identify suitable parameter settings for the machine learning models and to improve their classification performance. In this research, the

HalvingGridSearchCV method from the sklearn.model_selection library is used for hyperparameter optimization.

HalvingGridSearchCV evaluates different combinations of predefined hyperparameter values and progressively allocates computational resources to the more promising configurations. This approach reduces the computational cost of evaluating a large number of parameter combinations while identifying suitable model configurations.

Hyperparameter tuning is performed for the Decision Tree (DT), Random Forest (RF), XGBoost, and Multi-Layer Perceptron (MLP) models. The selected hyperparameters are determined during the training process using the training data. The test data are not used during hyperparameter optimization to avoid data leakage.

After tuning, the resulting model configurations are used for the final evaluation on the testing data. The performance of the tuned models is compared using the evaluation metrics defined in this research.

3.6.4 Categorical Encoding

Categorical encoding is performed to convert the target class labels into numerical values that can be processed by the machine learning models. In this research, both binary and multiclass classification are considered for the CICIoT2023 and EdgeIIoT datasets.

For binary classification, the target variable is represented by attack_flag, where **0** represents benign traffic and 1 represents malicious traffic**.** This representation enables the machine learning models to distinguish normal IoT traffic from attack traffic.

For multiclass classification, the attack categories are converted into numerical class labels. For the CICIoT2023 dataset, the classes include Benign, Brute Force, DDoS, DoS, Mirai, Reconnaissance, Spoofing, and Web-based attacks**.** For the EdgeIIoT dataset, the classes include Benign, DoS/DDoS, Man-in-the-Middle, Ransomware, Reconnaissance, and Web-based attacks**.**

The numerical class labels are used as the target variable during model training and evaluation. The same class-label representation is maintained throughout the corresponding experiments to ensure consistency in the classification process.

3.7Machine Learning Models

This research evaluates seven machine learning models for IoT intrusion detection. The selected models are Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF),

Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP).

These models are selected to provide a comparative evaluation of different machine learning approaches, including probabilistic, tree-based, distance-based, linear, gradient-boosting, and neural-network-based methods. Each model is trained using the prepared IoT network traffic datasets and evaluated for its ability to distinguish benign traffic from malicious traffic.

The experiments include both binary classification and multiclass classification. Binary classification distinguishes between benign and malicious traffic, while multiclass classification identifies different categories of attacks.

Hyperparameter tuning is performed for the selected models where applicable to improve their performance. The models are evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). The comparative results are used to identify the most effective machine learning model for the proposed IoT intrusion detection framework.

3.7.1 Gaussian Naive Bayes (GNB)

Gaussian Naive Bayes (GNB) is a probabilistic machine learning algorithm based on Bayes' theorem. It assumes that the features are conditionally independent given the class label and models continuous features using a Gaussian distribution.

In this research, GNB is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The model is trained using the preprocessed and selected network traffic features from the CICIoT2023 and EdgeIIoT datasets.

The performance of GNB is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). Its results are compared with the other six machine learning models to determine its effectiveness for IoT intrusion detection.

3.7.2 Decision Tree (DT)

Decision Tree (DT) is a supervised machine learning algorithm that uses a tree-like structure to make classification decisions. The model divides the input data into different branches based on feature values until a class prediction is obtained.

In this research, DT is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The model is trained using the preprocessed and selected features from the CICIoT2023 and EdgeIIoT datasets.

Hyperparameter tuning is applied to the Decision Tree model to identify suitable model parameters and improve its classification performance. The performance of DT is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR).
The results obtained from DT are compared with the other six machine learning models to determine its effectiveness for IoT intrusion detection.

3.7.3 Random Forest (RF)

Random Forest (RF) is an ensemble machine learning algorithm that combines multiple decision trees to perform classification. Each decision tree is trained using different subsets of the training data and features, and the final prediction is obtained by combining the predictions of the individual trees.
In this research, RF is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The model is trained using the preprocessed and selected features from the CICIoT2023 and EdgeIIoT datasets.
Hyperparameter tuning is applied to identify suitable parameters for the Random Forest model and improve its classification performance. The model is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR).
The performance of RF is compared with the other six machine learning models to determine its effectiveness for IoT intrusion detection.

3.7.4 Linear Support Vector Machine (LSVM)

Linear Support Vector Machine (LSVM) is a supervised machine learning algorithm that identifies a decision boundary for separating different classes of data. It aims to find an optimal hyperplane that maximizes the margin between classes.
In this research, LSVM is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The model is trained using the preprocessed and selected features from the CICIoT2023 and EdgeIIoT datasets.
The performance of LSVM is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). Its results are compared with the other six machine learning models to assess its effectiveness for IoT intrusion detection.

3.7.5 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm that classifies a data sample based on the classes of its nearest neighboring samples. The classification is determined by considering the specified number of neighboring data points according to a selected distance measure.

In this research, KNN is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The model is trained and evaluated using the preprocessed and selected features from the CICIoT2023 and EdgeIIoT datasets.

The performance of KNN is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). Its results are compared with the other six machine learning models to determine its effectiveness for IoT intrusion detection.

3.7.6 XGBoost

XGBoost (eXtreme Gradient Boosting) is an ensemble machine learning algorithm based on gradient boosting. It builds a sequence of decision trees, where each new tree is designed to improve the errors made by the previous trees.

In this research, XGBoost is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The model is trained using the preprocessed and selected features from the CICIoT2023 and EdgeIIoT datasets.

Hyperparameter tuning is applied to identify suitable parameters for the XGBoost model and improve its classification performance. The model is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR).

The performance of XGBoost is compared with the other six machine learning models. The comparative results are used to determine which model provides the most effective performance for IoT intrusion detection.

3.7.7 Multi-Layer Perceptron (MLP)

Multi-Layer Perceptron (MLP) is a feed-forward artificial neural network that consists of an input layer, one or more hidden layers, and an output layer. The model learns relationships between input features and target classes by adjusting its weights during the training process.

In this research, MLP is used to classify IoT network traffic into benign and malicious classes for binary classification and into different attack categories for multiclass classification. The

model is trained using the preprocessed and selected features from the CICIoT2023 and EdgeIIoT datasets.

Hyperparameter tuning is applied to identify suitable parameters for the MLP model and improve its classification performance. The model is evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR).

The performance of MLP is compared with the other six machine learning models to determine its effectiveness for IoT intrusion detection.

3.8 Model Training and Testing

After data preprocessing, feature selection, and feature scaling, the prepared datasets are used to train and test the seven selected machine learning models. The models include Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP).

The datasets are divided into using stratified sampling. The training data is used 70% training data and 30% testing data to train the machine learning models, while the testing data is kept separate and used to evaluate their performance. SMOTE is applied only to the training data to address class imbalance and avoid introducing synthetic samples into the testing set.

The experiments are conducted for both binary and multiclass classification. In binary classification, the models distinguish between benign and malicious traffic. In multiclass classification, the models classify network traffic into different attack categories.

Hyperparameter tuning is performed for the applicable models, including DT, RF, XGBoost, and MLP. The trained models are then evaluated on the testing data using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR).

The performance results of the seven models are compared to identify the most effective model for IoT intrusion detection. The selected model is subsequently considered for integration into the blockchain-based IoT security prototype described in the later stages of the methodology.

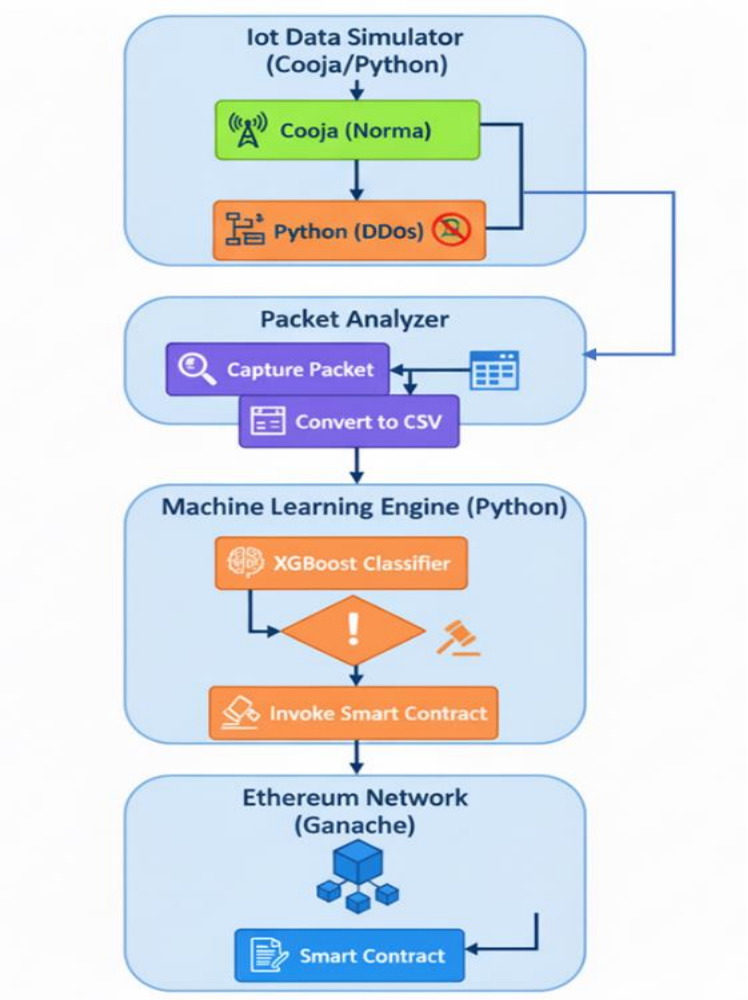


Figure 3.4 Flowchart for simulation of the Prototype

3.9 Model Evaluation Metrics

The performance of the seven machine learning models is evaluated using several performance metrics. These metrics provide a comprehensive assessment of the models' ability to correctly identify benign and malicious IoT network traffic.

Accuracy measures the proportion of correctly classified samples among all samples. It provides an overall indication of the classification performance of a model.

Precision measures the proportion of correctly identified positive samples among all samples predicted as positive. A higher precision indicates fewer false positive predictions.

Recall measures the proportion of actual positive samples that are correctly identified by the model. A higher recall indicates that the model is better at detecting malicious traffic.

F1-score is the harmonic mean of precision and recall. It provides a balanced measure of model performance, particularly when the dataset contains imbalanced classes.

False Positive Rate (FPR) measures the proportion of benign samples that are incorrectly classified as malicious. A lower FPR is desirable because excessive false alarms can reduce the effectiveness of an intrusion detection system.

Response Time measures the time required by the model to process and classify network traffic. Lower response time is desirable for IoT security applications where rapid detection and response are important.

These metrics are used to compare the seven machine learning models for both binary and multiclass classification. The results help identify the model that provides the most suitable balance between detection performance, false positives, and response time for the proposed IoT intrusion detection framework.

3.10 Blockchain-Based Prototype Development

The second phase of the research focuses on developing an end-to-end prototype that integrates the selected machine learning model with a permissioned Ethereum blockchain for IoT security. The purpose of this phase is to demonstrate how intrusion detection results can be used to support automated attack mitigation.

The IoT network is simulated using Cooja, where IoT devices communicate through the defined smart-city domains and their respective domain controllers. Wireshark is used to capture and analyze network traffic generated within the simulated environment.

The trained machine learning model analyzes the captured network traffic and identifies whether the traffic is benign or malicious. When malicious activity is detected, relevant information about the compromised device, such as its source IP address, is transferred to the blockchain layer.

A permissioned Ethereum blockchain is used to provide a controlled and trusted environment for managing IoT device information. The blockchain prototype is implemented in a local Ethereum development environment. Smart contracts are developed to manage device registration, access control, and automated security actions.

When a device is identified as malicious, the smart contract executes a predefined mitigation function. The compromised device can be removed or its access can be revoked from the trusted device registry. This process enables automated attack mitigation without requiring manual intervention.

The prototype demonstrates the integration of Machine Learning for intelligent intrusion detection with Blockchain for decentralized security management and automated response.

3.11 Cooja-Based IoT Network Simulation

Cooja is used in this research to simulate the proposed IoT network environment. The simulation provides a controlled environment in which IoT devices can communicate through the defined network domains and generate network traffic for security analysis.

The simulated network consists of IoT devices, sensors, actuators, and domain controllers representing different smart-city domains. The devices communicate through the simulated network, and their traffic is monitored to identify normal and potentially malicious communication.

Network traffic generated during the simulation is captured using Wireshark. The captured traffic provides information that can be analyzed by the machine learning-based intrusion detection component. The relevant traffic information is processed according to the preprocessing and feature-selection pipeline described earlier in this chapter.

When malicious traffic is identified by the selected machine learning model, information about the suspected device is forwarded to the blockchain layer. The blockchain-based smart contract then supports the predefined mitigation action, such as revoking or removing the compromised device from the trusted device registry.

The Cooja simulation therefore provides the IoT network environment required to demonstrate the integration of network traffic monitoring, machine learning-based intrusion detection, and blockchain-based automated mitigation.

3.12 Wireshark Traffic Capture and Analysis

Wireshark is used in this research to capture and analyze network traffic generated within the simulated IoT environment. The captured traffic provides the network-level information required for evaluating the proposed intrusion detection framework.

During the Cooja-based IoT network simulation, communication between IoT devices and domain controllers is monitored. Wireshark captures the resulting network packets and provides information about the communication flows and network activities.

The captured traffic is processed to obtain the relevant network traffic features required by the machine learning models. The extracted information is subjected to the preprocessing, feature-selection, and feature-scaling procedures described in the previous sections.

The prepared network traffic is then provided to the selected machine learning model for classification. The model identifies whether the observed traffic represents benign or malicious

activity. When malicious activity is detected, information about the suspected device is forwarded to the blockchain-based security layer for automated mitigation.

This process connects the IoT network simulation with the machine learning intrusion detection component and the blockchain-based mitigation mechanism, supporting the development of the proposed end-to-end security prototype.

3.13 Blockchain and Smart Contract Implementation

A permissioned Ethereum blockchain is used in the proposed prototype to provide decentralized management of IoT device information and security-related actions. The blockchain component is implemented in a local development environment to demonstrate the interaction between the machine learning intrusion detection component and the blockchain layer.

Smart contracts are developed to manage the registration and access status of IoT devices. Authorized devices are added to the trusted device registry, while information related to compromised or unauthorized devices can be updated through predefined smart-contract functions.

When the machine learning component detects malicious activity, the relevant device information, such as the source IP address, is forwarded to the blockchain layer. The smart contract verifies the device information and executes the predefined mitigation action. Depending on the security status, the compromised device can be removed from the trusted registry or have its access revoked.

The use of smart contracts enables the mitigation process to be performed automatically according to predefined rules. This reduces the need for manual intervention and provides a transparent record of security-related actions within the blockchain environment.

The blockchain component therefore complements the machine learning-based intrusion detection system by providing decentralized device management and automated attack mitigation.

3.14 Automated Attack Mitigation

The automated attack mitigation component is designed to respond to malicious activity detected by the machine learning-based intrusion detection system. When the selected machine learning model classifies network traffic as malicious, relevant information about the suspected IoT device is transferred to the blockchain layer.

The device information, such as the source IP address, is provided to the smart contract. The smart contract checks the security status of the device and executes the predefined mitigation function when the device is identified as compromised.

The mitigation mechanism removes the compromised device from the trusted device registry or revokes its access to the IoT network. This prevents the identified malicious device from continuing to participate as a trusted device in the proposed environment.

The automated response reduces the need for manual intervention and provides a systematic mechanism for responding to detected attacks. The process demonstrates how Machine Learning and Blockchain can work together, where Machine Learning performs intelligent intrusion detection and Blockchain smart contracts support automated security actions.

The effectiveness of the mitigation mechanism is evaluated based on the successful detection and response to malicious devices, as well as the response time of the proposed system.

3.15 Overall Proposed Framework

The overall proposed framework integrates the machine learning-based intrusion detection component with the blockchain-based security and automated mitigation components. The framework follows a sequential process from IoT network simulation and traffic collection to data preprocessing, feature selection, machine learning classification, blockchain-based device management, and automated attack mitigation.

First, the IoT network is simulated using Cooja, and network traffic is captured using Wireshark. The captured traffic is prepared through data preprocessing, feature selection, and feature scaling. The prepared data is then used to train and evaluate seven machine learning models: GNB, DT, RF, LSVM, KNN, XGBoost, and MLP.

The models are evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate. The comparative results are used to identify the most effective model for the proposed intrusion detection component.

The selected model is then integrated with the blockchain-based prototype. When malicious activity is detected, information about the suspected IoT device is forwarded to the permissioned Ethereum blockchain. Smart contracts manage the device status and execute predefined mitigation actions, such as removing or revoking access for compromised devices.

The overall framework demonstrates an end-to-end approach in which Machine Learning performs intrusion detection, while Blockchain and smart contracts support decentralized security management and automated attack mitigation.

3.16 Research Methodology Summary

This chapter presented the methodology used to develop and evaluate the proposed Machine Learning and Blockchain-based intrusion detection framework for IoT environments. The methodology consists of two major phases.

The first phase focuses on the comparative evaluation of seven machine learning models using the CICIoT2023 and EdgeIIoT datasets. The datasets undergo preprocessing, class-imbalance handling, feature selection, feature scaling, model training, and testing. The models are evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate.

The second phase focuses on the development of an end-to-end blockchain-based IoT security prototype. Cooja is used for IoT network simulation, while Wireshark is used for network traffic capture and analysis. The selected machine learning model is integrated with a permissioned Ethereum blockchain and smart contracts to support device management and automated attack mitigation.

The proposed methodology provides a systematic approach for addressing the research objectives and questions. It combines intelligent intrusion detection with decentralized security management and automated mitigation to improve the security of IoT environments.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the experimental results obtained from the evaluation of the proposed Machine Learning and Blockchain-based intrusion detection framework for IoT environments. The experiments are conducted using the CICIoT2023 and EdgeIIoT datasets and the seven selected machine learning models: Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP).

Section 4.2 presents the experimental setup and computing environment used to conduct the experiments. Section 4.3 presents the exploratory data analysis of the CICIoT2023 and EdgeIIoT datasets. Section 4.4 describes the data preprocessing, sampling, and feature-selection results. Section 4.5 presents the binary and multiclass classification results of the seven machine learning models. The models are compared using accuracy, precision, recall, F1-score, response time, and false positive rate.

Section 4.6 presents the blockchain-based IoT security prototype and demonstrates the integration of the selected machine learning model with a permissioned Ethereum blockchain and smart contracts for automated attack mitigation. The chapter concludes by summarizing the experimental findings and the validation of the research hypotheses.

4.2 Experimental Setup

The experiments were conducted using a local computing environment consisting of a laptop and the software tools required for machine learning, network traffic analysis, and blockchain prototype development. The hardware specifications, operating system, programming environment, and software versions used in the experiments are presented in Table 4-1.

The machine learning experiments were implemented using Python and the required machine learning libraries. Wireshark was used for network traffic capture and analysis, while Truffle and Ganache were used for the development and testing of the Ethereum-based blockchain prototype. Solidity and Web3.js were used to implement and interact with the smart contracts.

Table 4-1 Testbed Environment Setup Used in The Praxis

HARDWARE	COMPUTER	APPLE MACBOOK PRO
	Chip	Apple M3 Pro
	Memory	36GB
	Operating System	macOS Sonoma version 14.4.1

SOFTWARE	Python (Anaconda3)	3.9.15
	Visual Studio Code	1.87.2
	Wireshark	v4.2.4
	Truffle	v5.11.4
	Ganache v7.9.1	v7.9.1
	Solidity	0.8.13 (solc-js)
	Node	v18.17.1
	Web3.js	v1.10.0

The experimental environment was configured to support both the comparative evaluation of the seven machine learning models and the development of the blockchain-based IoT security prototype.

4.3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to examine the characteristics, distribution, and structure of the CICIoT2023 and EdgeIIoT datasets before applying the machine learning models. EDA helps identify class distributions, feature characteristics, potential outliers, and data imbalance that may affect the subsequent analysis.

In this research, EDA is conducted on both datasets to understand the distribution of benign and malicious network traffic and to examine the statistical characteristics of the available network traffic features. The results of the EDA are used to guide the data preprocessing, sampling, and feature-selection procedures.

The following subsections present the EDA results for the CICIoT2023 and EdgeIIoT datasets.

4.3.1 CICIoT2023 Dataset

The CICIoT2023 dataset shows a highly imbalanced distribution of network traffic classes, as illustrated in Figure 4-1. The distribution indicates that some attack categories contain substantially more samples than others, while benign traffic represents a smaller proportion of the dataset. This class imbalance can affect machine learning performance, particularly for minority attack categories, and therefore requires appropriate preprocessing and sampling techniques.

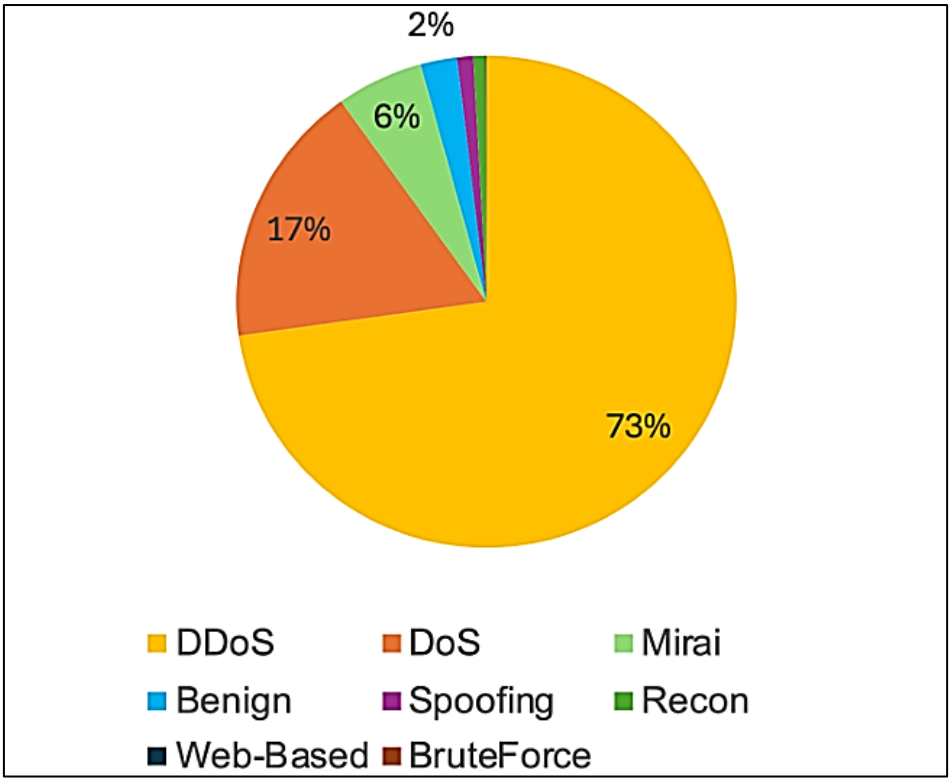


Figure 4.1 CICIoT Attack Vs Benign Distribution

Table 4-2 shows some statistics of the CICIoT dataset features. These attributes include boolean: HTTP, HTTPS, and Domain Name System (DNS), which represent the protocol that is used for the traffic. Also, states of TCP connections are represented by the flags fin_flag_number, syn_flag_number, and rst_flag_number. Also, we calculate skewness and kurtosis of each feature. The measure of asymmetry of the distribution is skew. Kurtosis is a measure of the skewness of the distribution. A high kurtosis means heavy-tailed (also known as outliers), whereas a low kurtosis means light-tailed (also known as no outliers) (Kim, 2013). Several features (e.g., flow_duration, Header_Length, Rate, Srate, Drate, and Covariance) have high kurtosis and are skewed. These statistics will be a part of the feature selection procedure.

Table 4-2 Summary Statistics for CICIoT Dataset Features

FEATURE	MEAN	STD	MIN	25%	50%	75%	MAX	KURTOSIS	SKEWNESS
FLOW_DURATION	5.8	285.1	0	0	0	0.1	394,357.20	141,887.80	240.2
HEADER_LENGTH	461,317.20	461,317.2 0	0	54	54	280.5	9,907,147. 80	118	9.7
PROTOCOL TYPE	9.1	8.9	0	6	6	14.3	47	8.3	2.7
DURATION	66.4	14	0	64	64	64	255	78	8.3

RATE	9,065.30	99,584.20	0	2.1	15.8	117.4	8,388,608.00	699.8	22.8
SRATE	9,065.30	99,584.20	0	2.1	15.8	117.4	8,388,608.00	699.8	22.8
DRATE	0	0	0	0	0	0	29.7	11,641,328.10	3,185.70
FIN_FLAG_NUMBER	0.1	0.3	0	0	0	0	1	6.6	2.9
SYN_FLAG_NUMBER	0.2	0.4	0	0	0	0	1	0.1	1.4
RST_FLAG_NUMBER	0.1	0.3	0	0	0	0	1	6.1	2.9
PSH_FLAG_NUMBER	0.1	0.3	0	0	0	0	1	6.5	2.9
ACK_FLAG_NUMBER	0.1	0.3	0	0	0	0	1	3.2	2.3
ACK_COUNT	0.1	0.3	0	0	0	0	7.1	7.6	3
SYN_COUNT	0.3	0.7	0	0	0	0.1	12.9	5.1	2.2
FIN_COUNT	0.1	0.3	0	0	0	0	248.3	15,105.80	44.6
URG_COUNT	6.2	71.9	0	0	0	0	4,401.70	717.8	24
RST_COUNT	38.5	325.4	0	0	0	0	9,613.00	202.6	12.8
HTTP	0	0.2	0	0	0	0	1	15.8	4.2
HTTPS	0.1	0.2	0	0	0	0	1	13.2	3.9
DNS	0	0	0	0	0	0	1	7,649.10	87.5
TELNET	0	0	0	0	0	0	1	46,467,774.00	6,816.70
SMTP	0	0	0	0	0	0	1	15,489,254.70	3,935.60
SSH	0	0	0	0	0	0	1	24,477.50	156.5
IRC	0	0	0	0	0	0	1	6,638,249.10	2,576.50
TCP	0.6	0.5	0	0	1	1	1	-1.9	-0.3
UDP	0.2	0.4	0	0	0	0	1	0	1.4
DHCP	0	0	0	0	0	0	1	580,842.20	762.1
ARP	0	0	0	0	0	1	15,091.70	15,091.70	122.9
ICMP	0.2	0.4	0	0	0	1	1.3	1.3	1.8
IPV	1	0	0	1	1	1	8,862.90	8,862.90	-94.2
LLC	1	0	0	1	1	1	8,862.90	8,862.90	-94.2

TOT SUM	1,308.30	2,613.40	42	525	562	56.5	127,335.80	57.9	5.9
MIN	91.6	139.7	42	50	54	54	13,583.00	46.9	5.1
MAX	182	524	42	50	54	55.2	49,014.00	164.4	9.5
AVG	124.7	241	42	50	54	54	13,583.00	42	5.1
STD	33.3	160.3	0	0	0	0.4	12,385.20	120	8.3
TOT SIZE	124.7	241.5	42	50	54	54.1	13,583.00	38.4	5
IAT	83,182,559.70	17,047,224.30	0	83,071,565.80	83,124,522.40	83,343,907.90	167,639,436.00	20.8	0
NUMBER	9.5	0.8	1	9.5	9.5	9.5	15	21.2	-0.1
MAGNITUE	13.1	8.6	9.2	10	10.4	10.4	164.8	10.3	3.1
RADIUS	47.1	226.7	0	0	0	0.5	17,551.30	120.4	8.3
COVARIANCE	30,717.2 0	323,553.7 0	0	0	0	1.3	154,902,159.20	9,821.20	60.4
VARIANCE	0.1	0.2	0	0	0	0.1	1	8.1	3
WEIGHT	141.5	21.1	1	141.6	141.6	141.6	244.6	21	0

Data profiling is done to explore and examine the correlation between the protocol and type of attack as shown in Figure 4-2. Interestingly, some attacks seem to be more associated with protocols. An example of this is the Internet Control Message Protocol (ICMP) which is mainly used for DDoS ICMP Flood attacks. Over half the DDoS attacks are from the User Datagram Protocol (UDP). About half of the traffic on the Logical Link Control (LLC) protocol is command injection attacks. In contrast, Hypertext Transfer Protocol (HTTP), Hypertext Transfer Protocol Secure (HTTPS), and Transmission Control Protocol (TCP) all are more varied and are less specific to attacks. This separation occurs because they are used in various situations of data transfer.

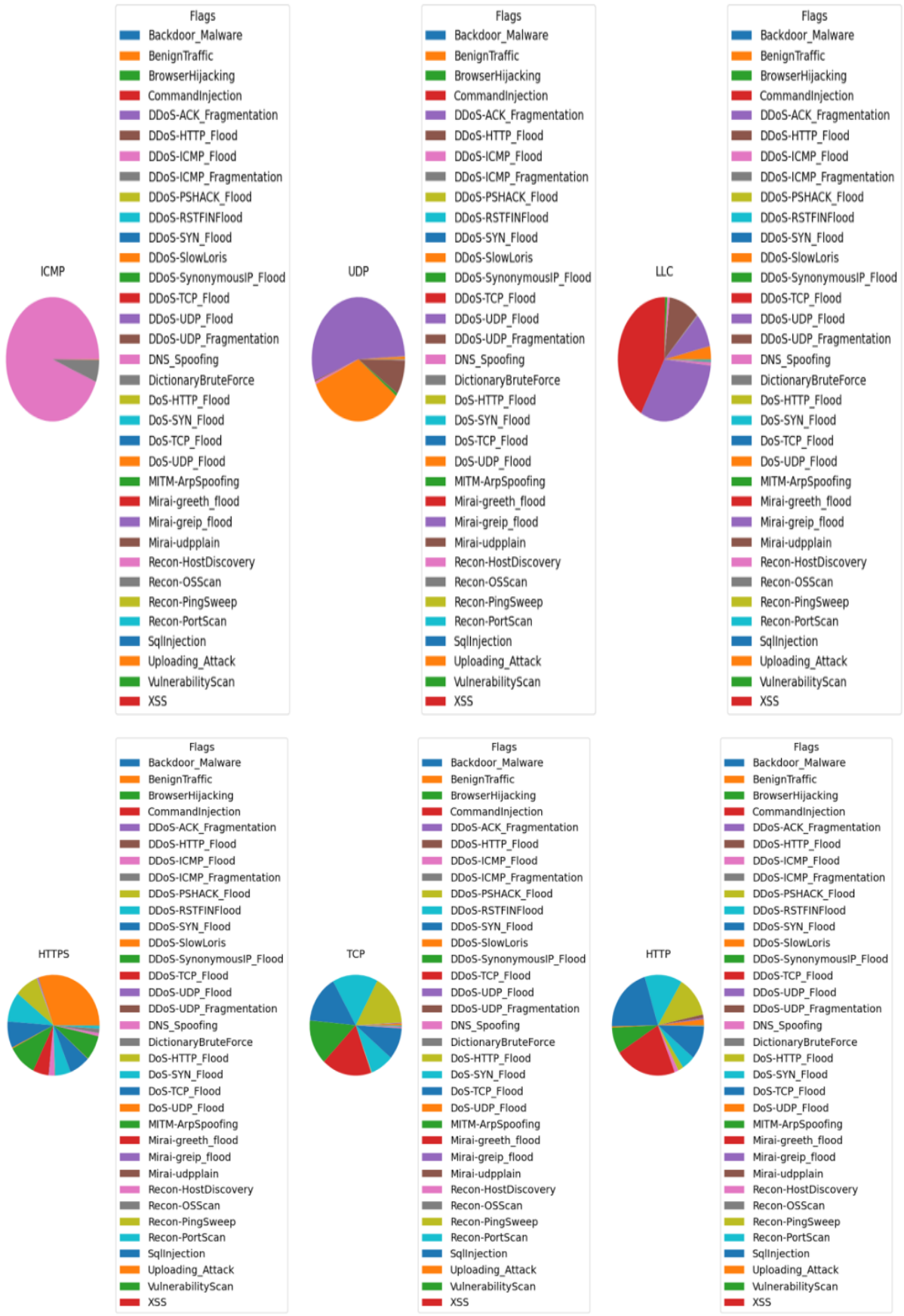


Figure 4.2 CICIoT Protocols Vs Attack Distribution

4.3.2 EdgeIIoT Dataset

Exploratory Data Analysis was also performed on the EdgeIIoT dataset to examine its class distribution and statistical characteristics. The dataset contains benign network traffic as well as multiple categories of malicious activities. The distribution of the traffic classes is examined to identify differences in the number of samples among the classes.

The analysis indicates that the EdgeIIoT dataset contains an imbalanced distribution across its traffic classes. Some attack categories contain considerably more samples than others, which may influence the performance of machine learning models during classification. Therefore, appropriate preprocessing and class-imbalance handling techniques are applied before model training.

The statistical characteristics of the network traffic features are also examined to understand their distributions and identify features that may require further processing. The EDA results provide a basis for the subsequent preprocessing, feature-selection, and machine learning experiments.

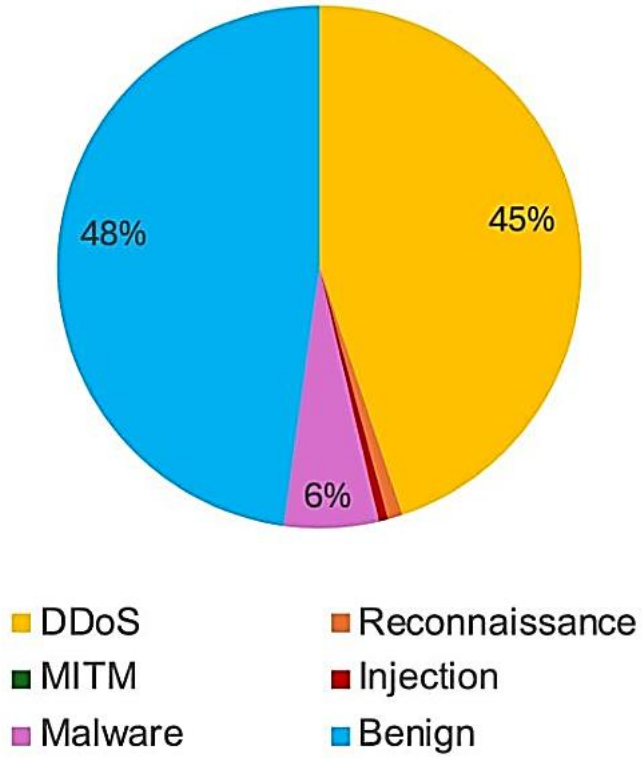


Figure 4.3 EdgeIIoT Attack Vs Benign Distribution

The EdgeIIoT dataset comes from various sources and contains attributes for various protocols such as HTTP, ARP, ICMP, MQTT, TCP, UDP, DNS, IPv4, and Modbus/TCP (Ferrag et al., 2022). Interestingly, some of the features are not useful for machine learning evaluation, such

as ‘icmp.unused’ and 'mqtt.msg_decoded_as.' Further, IP addresses and protocol information may be ignored when evaluating, and smart contracts are one way IP addresses can be used in attacks and mitigated. In addition, we examqine skewness and kurtosis for each feature. What is interesting is that the features whose shapes are skewed, such as 'http.content_length,' 'tcp.connection.syn,' 'dns.retransmit_request,' and 'dns.retransmit_request_in,' have a higher kurtosis than 3. For comparability, we rely on a data conversion tool that is supplied by Python to convert the raw PCAP data to a set of features that are also supplied by CICIoT.

Table 4-3 Summary Statistics for EdgeIIoT Dataset Features

FEATURES	MEAN	STD	MIN	25%	50%	75%	MAX	KURTOSIS	SKEWNESS
ICMP.UNUSED	0	0	0	0	0	0	0	0	0
HTTP.CONTENT_LENGTH	4.8	96.3	0	0	0	0	83,655.00	305,218.80	359.8
HTTP.RESPONSE	0	0.1	0	0	0	0	1	124.1	11.2
HTTP.TLS_PORT	3,529.20	70,057.20	0	0	1	0	6,939,784.00	3,470.40	47.8
TCP.ACK	89,464,927. 60	391,193,225. 20	0	0	1	15	4,274,334,010. 00	35.7	5.5
TCP.CONNECTION.RST	0.1	0.3	0	0	0	0	1	4.3	2.5
TCP.CONNECTION.SYN	0.1	0.8	0	0	0	0	1,440.00	2,756,159.10	1,539.5 0
TCP.CONNECTION.SYNACK	190.1	3,333.70	0	0	0	0	64,666.00	327.5	18
TCP.FLAGS.ACK	1	23.8	0	0	1	1	5,900.00	4,546.60	62.6
TCP.SEQ	1,037,707.6 0	12,296,557.7 0	0	0	1	6	3,302,298,838. 00	2,821.40	23.9
UDP.STREAM	225,812.60	620,122.90	0	0	0	0	2,898,784.00	6.9	2.8
UDP.TIME_DELTA	0	4.1	0	0	0	0	876.6	10,819.20	101.1
DNS.QRY.TYPE	0	0.3	0	0	0	0	37	11,309.40	106.1
DNS.RETRANSMISSION	0	0.2	0	0	0	0	28	19,164.50	138.1
DNS.RETRANSMIT_REQUES T	0	0	0	0	0	0	1	219,340.40	468.3

DNS.RETRANSMIT_REQUES T_IN	0.2	169.4	0	0	0	0	146,274.00	745,769.40	863.6
MQTT.CONFLAG.CLEANSESS	0	0.2	0	0	0	0	12	115.1	6.8
MQTT.MSG_DECODED_AS	0	0	0	0	0	0	0	0	0
MQTT.MSGTYPE	0.6	2.4	0	0	0	0	14	26.6	5.2
MQTT.TOPIC_LEN	0.4	2.3	0	0	0	0	24	59.1	7.3
MQTT.VER	0.2	2.2	0	0	0	0	187	350.8	17.6
MBTCP.TRANS_ID	0.1	1.3	0	0	0	0	151	480.2	16.6
MBTCP.UNIT_ID	0	0	0	0	0	0	6	644,000.10	706.4
ATTACK_LABEL	0.5	0.5	0	0	1	1	1	-2	-0.1
ARP.OPCODE	Nan	nan	nan	nan	Nan	Nan	Nan	11,913.20	97.3
ICMP.TRANSMIT_TIMESTA MP	Nan	Nan	Nan	Nan	Nan	Nan	Nan	233,058.40	482.8
UDP.PORT	nan	Nan	Nan	Nan	Nan	Nan	Nan	8,331.20	90.1
MQTT.LEN	nan	Nan	Nan	Nan	Nan	Nan	Nan	29.7	5.2
MQTT.PROTO_LEN	Nan	Nan	Nan	Nan	Nan	Nan	Nan	30.6	5.7
MBTCP.LEN	Nan	nan	Nan	nan	nan	Nan	Nan	342.4	18

4.4 PCAP to CSV Data Conversion

The two datasets contain captured PCAP files, with different features in their CSV files. The features for EdgeIIoT are directly extracted from the network traffic analysis with TShark and Zeek tools, and the features for CICIoT are extracted with the custom Python scripts. Scripts to extract the number of packets, average rate, and average duration from the PCAP files. The EdgeIIoT PCAP data is converted to be consistent with features offered by CICIoT using a customized Python program. This conversion can be used to compare the features in order to standardize the characteristics to compare, especially for evaluating hypothesis 1 for the effectiveness of features in intrusion attack classification.

4.5 Data Preprocessing and Feature Selection Results

This section presents the results obtained after preprocessing the CICIoT2023 and EdgeIIoT datasets and applying the feature-selection techniques described in Chapter 3. The preprocessing stage prepares the datasets for machine learning experiments, while feature selection identifies the most relevant network traffic features for intrusion detection.

The feature-selection analysis considers Pearson Correlation Coefficient, Variance Threshold, Mutual Information, Chi-Square Test, and the Combined Method. The resulting feature sets are compared to determine the features that contribute most effectively to the classification of benign and malicious IoT network traffic.

The selected features are subsequently used in the evaluation of the seven machine learning models. The results are presented using the corresponding tables and figures in the following subsections.

4.5.1 Data Cleaning

The data-cleaning process was performed to prepare the CICIoT2023 and EdgeIIoT datasets for machine learning experiments. Duplicate records were removed, and missing values, including NaN (Not a Number) values, were identified and handled. These steps helped improve data quality and ensured that the datasets were suitable for subsequent preprocessing, feature selection, and machine learning model training.

4.5.2 Data Splitting and Class Balancing

The preprocessed datasets were divided into training and testing sets using a 70:30 ratio. The training set was used to train the machine learning models, while the testing set was used to evaluate their performance on unseen data. To address class imbalance in the training data, the Synthetic Minority Over-sampling Technique (SMOTE) was applied.

Random sampling, as in (1) is a method of selecting a subset of instances randomly. To prevent overloading the environment, 20% of the time, the entire data set is sampled at random for uploading to the environment.

$andas.DatFrame.(frac = 0.2, random_state = 42)$

In (2) a stratified sample is obtained for certain stratifications. It ensures that the distribution of every class in the training and test sets is the same as in the original set. This helps maintain the distribution of the target variable, in turn resulting in more reliable and generalizable models.

$klearn.model_selection.train_test_(X,Y,test_size = 0.3,stratify = Y,random_state =$ 42)

Another sampling method used in the research is the Synthetic Minority Oversampling Technique (SMOTE) which is shown in (3). Imbalances can cause sub-optimal performance, particularly for minorities. To overcome the problem of imbalanced dataset, SMOTE is a technique of oversampling the minority class. In the category allocation, one sample of the category is shown before and after SMOTE sampling in Figure 4-4. The data set is relatively more balanced from the chart but some data contains more than 90% of the total data for instance brute force, DDoS, Mirai etc.

$mblearn.over_sampling.(sampling_strategy = minority',random_state =$ 42,$k_neighbors = 5,n_jobs = None$)

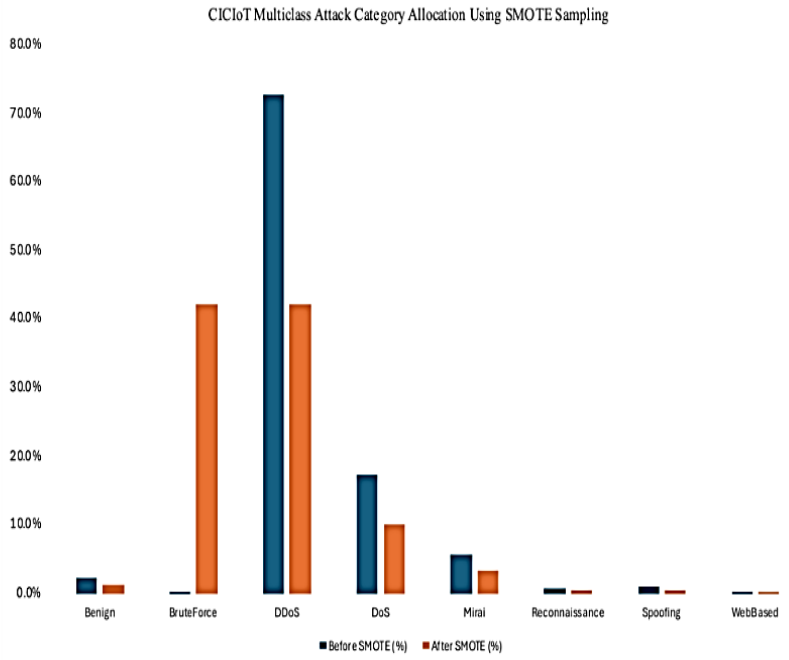


Figure 4.4 CICIoT Multiclass Attack Category Allocation Using SMOTE Sampling

4.6 Feature Engineering

4.6.1 Feature Selection

Feature selection was performed to identify the most informative network traffic features for IoT intrusion detection. Five feature-selection approaches were considered:

Pearson Correlation Coefficient, Variance Threshold, Mutual Information, Chi-Square Test, and the Combined Method. The individual techniques were used to evaluate the relevance and

characteristics of the available features, while the Combined Method was used to obtain a final set of 14 relevant features.

Pearson Correlation Coefficient

The Pearson correlation coefficient is a measure of linear association that is used to determine the strength of a linear relationship between two quantities and ranges from '-1' to '1'.

The Pearson correlation coefficient measures the strength and direction of the linear relationship between two variables and ranges from −1 to +1. A positive value indicates a positive relationship, while a negative value indicates a negative relationship. A value close to zero indicates a weak or negligible linear relationship.

When the value is '0' it means there is no correlation between two variables (Shardlow, 2016). The correlation between each feature and the attack flag was analyzed to determine the relevance of the features for classification.

Figure 4-5 displays the correlation coefficient of each of the features with the attack flag. The Pearson correlation coefficient measures the linear relationship between each feature and the attack flag. Features with stronger absolute correlation values indicate a stronger linear association with the target variable. The analysis also identified features such as Duration and rst_count with correlation coefficients close to −0.5.

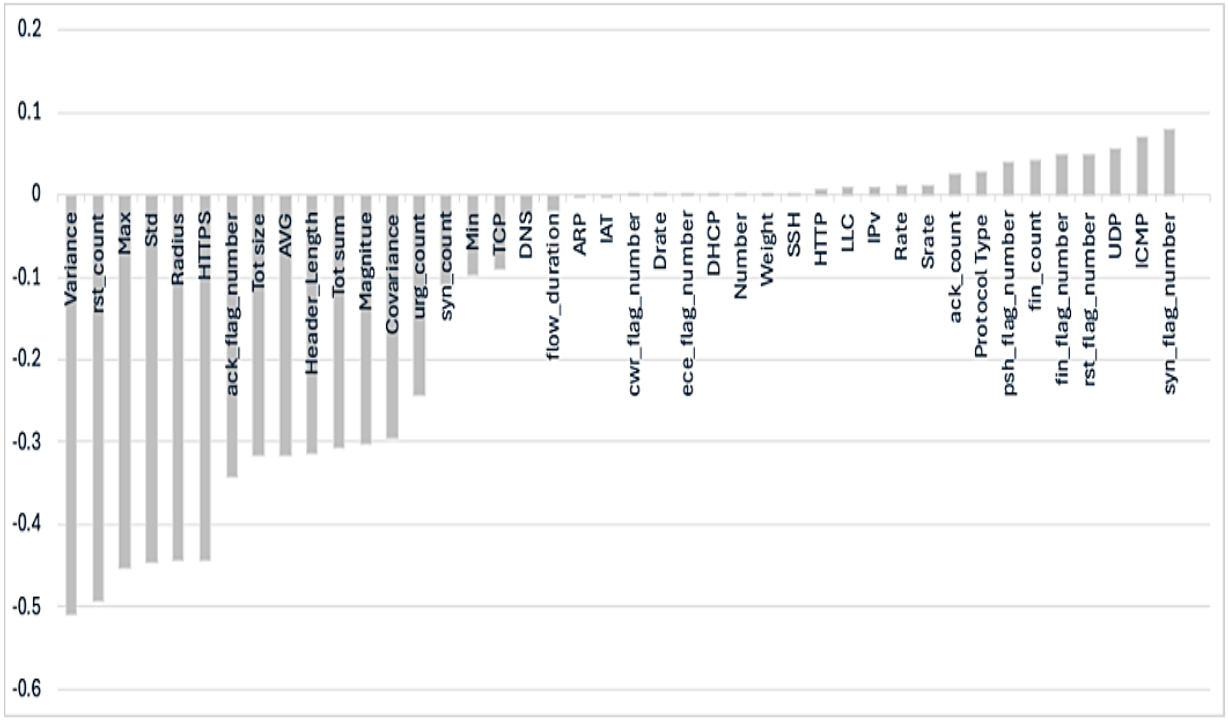


Figure 4.5 Pearson Correlation Coefficient Attack Flag (CICIoT)

But there are also some features (such as “Duration” and “rst_count”) with correlation coefficients near ‘-0.5’. This correlation matrix along with the skewness indicator from exploratory data analysis, helps in feature selection.

Variance Threshold

Variance Threshold is a feature-selection method that removes features with variance below a specified threshold.

Variance threshold filters out features with a variance below a given threshold. It will automatically drop all features that have no variation by default. Variance Threshold is a feature-selection method that removes features whose variance is below a specified threshold. In this study, a threshold of $0.8 \times (1 - 0.8)$ was applied, resulting in 24 selected features, as shown in Figure 4-6.

Mutual Information

Mutual Information measures the statistical dependence between a feature and the target variable. Higher mutual-information values indicate stronger dependence between the feature and the target. The Mutual Information method identified 20 relevant features.

Chi-Square Test

The Chi-Square Test evaluates the dependence between non-negative feature values and the target class. Features with stronger Chi-Square scores are considered more relevant for classification. The Chi-Square Test identified 20 relevant features.

#	Original CSV File - 47 Features	Variance Threshold - 24 Features	Mutual Information - 20 Features	Chi-Square - 20 Features	Combined Method - 14 Features
1	Duration	flow_duration	flow_duration	flow_duration	flow_duration
2	Variance	Header_Length	Header_Length	Header_Length	Header_Length
3	rst_count	Protocol Type	Protocol Type	Protocol Type	Duration
4	Max	Duration	Rate	Duration	Rate
5	Std	Rate	Srate	Rate	Srate
6	Radius	Srate	syn_flag_number	Srate	Tot sum
7	HTTPS	syn_flag_number	syn_count	fin_flag_number	Min
8	ack_flag_number	syn_count	urg_count	syn_count	Max
9	Tot size	urg_count	rst_count	urg_count	AVG
10	AVG	rst_count	TCP	rst_count	Std
11	Header_Length	TCP	UDP	Tot sum	Tot size
12	Tot sum	UDP	ICMP	Min	Magnitue
13	Magnitue	Tot sum	Tot sum	Max	Radius
14	Covariance	Min	Min	AVG	Covariance
15	urg_count	Max	Max	Std	
16	syn_count	AVG	AVG	Tot size	
17	Min	Std	Std	IAT	
18	TCP	Tot size	Tot size	Magnitue	
19	DNS	IAT	IAT	Radius	
20	flow_duration	Number	Magnitue	Covariance	
21	ARP	Magnitue			
22	IAT	Radius			
23	cwr_flag_number	Covariance			
24	Drate	Weight			
25	ece_flag_number				
26	DHCP				
27	Number				
28	Weight				
29	SSH				
30	HTTP				
31	LLC				
32	IPv				
33	Rate				
34	Srate				
35	ack_count				
36	Protocol Type				
37	psh_flag_number				
38	fin_count				
39	fin_flag_number				
40	rst_flag_number				
41	UDP				
42	ICMP				
43	syn_flag_number				
44	Telnet				
45	SMTP				
46	IRC				
47	timestamp				

Combined Method

The Combined Method integrates the results of the four feature-selection techniques to obtain the final 14 most relevant features, as illustrated in Figure 4-6. The correlation coefficients of the selected 14 features are shown in Figure 4-7.

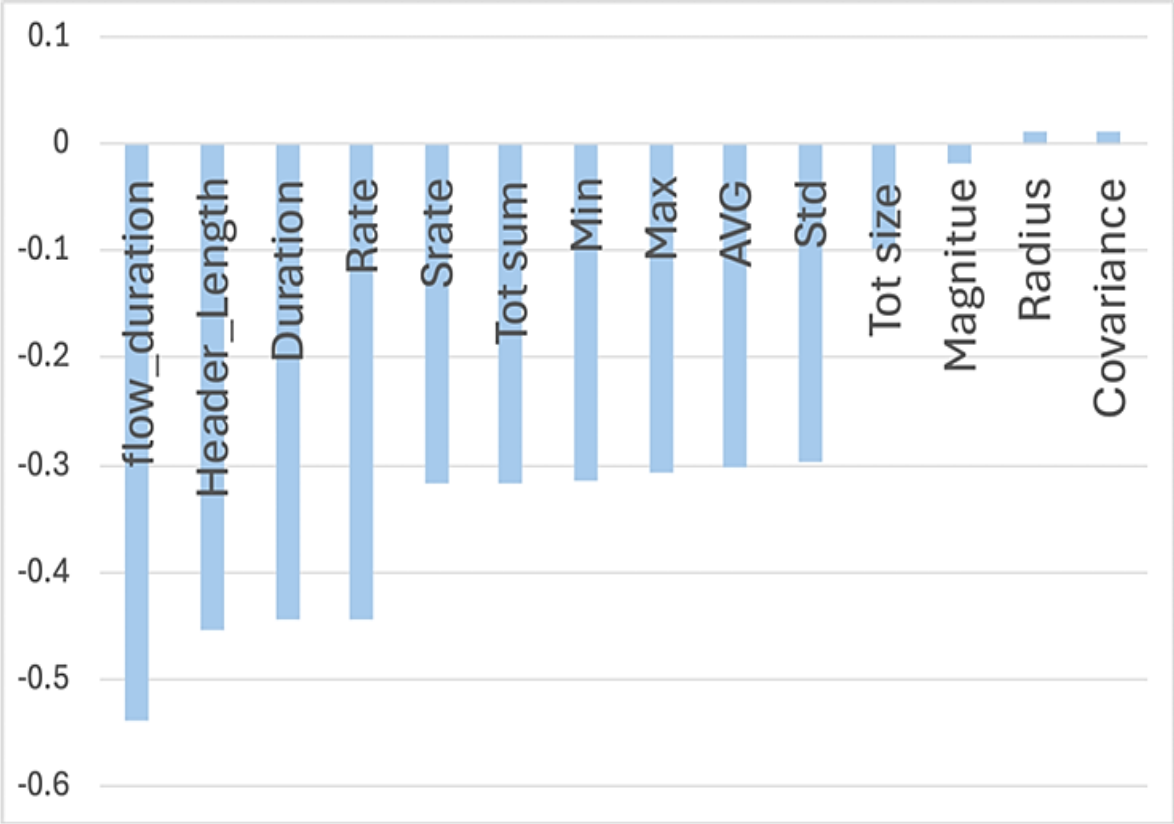


Figure 4.6 Correlation Coefficient of Selected 14 Features

Combined Method

The Combined Method integrates the results of the four feature-selection techniques to obtain the final 14 most relevant features, as illustrated in Figure 4-6. The correlation coefficients of the selected 14 features are shown in Figure 4-7.

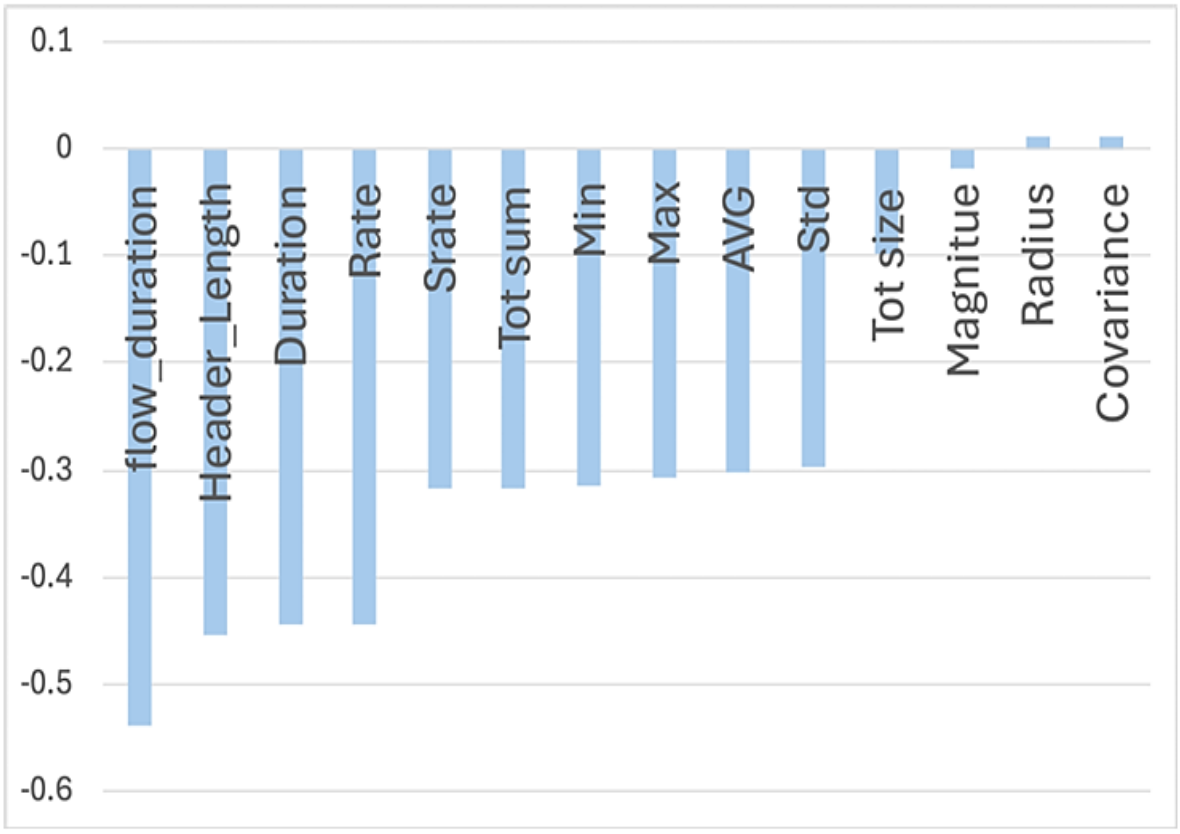


Figure 4.7 Correlation Coefficient of Selected 14 Features

4.6.2 Hyperparameter Tuning

Hyperparameter tuning was performed to identify suitable parameter configurations for the machine learning models and improve their classification performance. The tuning process was performed using HalvingGridSearchCV, which systematically evaluates candidate parameter combinations while progressively allocating computational resources to the most promising configurations.

In this research, hyperparameter tuning was applied to Random Forest (RF), Decision Tree (DT), XGBoost, and Multi-Layer Perceptron (MLP). Gaussian Naive Bayes (GNB), Linear Support Vector Machine (LSVM), and K-Nearest Neighbors (KNN) were evaluated using their defined/default configurations.

The hyperparameter configurations used in the experiments are presented in Table 4-4. These configurations were used during the machine learning evaluation to obtain the reported classification and response-time results.

Table 4-4 Hyperparameters Defined in Each Classifier

CLASSIFIER	HYPERPAMETERS

GAUSSIAN NAÏVE BAYES	Default
RANDOM FOREST	max_depth=10, min_samples_split=10, n_estimators=24, random_state=42
DECISION TREE	criterion="entropy", random_state=42, max_depth=3, min_samples_leaf=5
XGBOOST	gamma=0.5, max_depth=6
MULTI-LAYER PERCEPTRON	random_state=1, max_iter=300, hidden_layer_sizes=150, solver='adam'
LINEAR SUPPORT VECTOR MACHINE	Default
K-NEAREST NEIGHBORS	n_neighbors=3

4.7 Machine Learning Model Evaluation

The machine learning models were evaluated using the CICIoT2023 and EdgeIIoT datasets. Seven machine learning models, namely Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP), were evaluated for IoT intrusion detection.

The evaluation was performed for both binary and multiclass classification. Binary classification distinguishes benign traffic from malicious traffic, while multiclass classification identifies different categories of attacks. The evaluation was performed using accuracy, precision, recall, F1-score, response time, and false positive rate (FPR). The results are presented in the following subsections and are used to address the research questions and evaluate the corresponding hypotheses.

4.7.1 Binary Classification Measurement Metrics

Binary classification was performed to distinguish between benign and malicious IoT network traffic. The seven machine learning models were evaluated using the CICIoT2023 and EdgeIIoT datasets. The evaluation considered accuracy, precision, recall, and F1-score as the primary classification performance measures.

The results obtained from the two datasets are presented in the following tables. The performance of the seven models is compared to identify the models that provide the most effective classification of benign and malicious IoT traffic.
below.

$$Accuracy = \frac{(True\ Positive + True\ Negative)}{(True\ Positive + True\ Negative + False\ Positive + False\ Negative)} \quad (4)$$

$$Average\ Accuracy = \Sigma\big(Accuracy(i)\big) \div 3$$

$$(\mathrm{i} = 47, 20, \text{or } 14)$$

Feature Significance by Standard Deviation of Accuracy

The feature-significance analysis evaluates whether a smaller subset of network traffic features can provide effective intrusion classification. Three feature sets containing 47, 20, and 14 features were evaluated. The standard deviation of accuracy was calculated to compare the consistency of the classification results across these feature sets.To check the performance of various sets of features, we perform tests on the datasets by using three different sets of features: 47 features, 20 features, and 14 features. In those scenarios, we calculate the standard deviation of the accuracy as shown in the formula below. The standard deviation of accuracy in (5) indicates how much deviation to expect from the average accuracy. If this value is low, then all of the accuracies are grouped around the mean. The results indicate that the selected 14 features provide the most consistent indication for intrusion detection.

$$\text{Standard Deivation of Accuracy} = \frac{\sqrt{\Sigma(\text{Accuracy}(i) - \text{Mean Accuracy})^2}}{3} \quad (5)$$

$$(\mathrm{i} = 47, 20, \text{or } 14)$$

Overall Performance

To measure the overall performance of each classifier, the average accuracy in (6), the average precision in (7) and the average F1 score in (8) are used.

$verage\ of\ Precision = \Sigma(Precision(i) \div 3$ (6)

$verage\ of\ Recall = \Sigma(Recall(i) \div 3$ (7)

$verage\ of$ F1-score $= \Sigma(F1Score(i) \div 3$ (8)

(i = 47,20,or 14)

False Positive Rate

As with accuracy and overall performance, the average false positive rate (9) is calculated over three tests with 47, 20 and 14 features. This measure will give an overview of performance for all classifications.

$$verage\ of\ FalsePositiveRate = \Sigma(FalsePositiveRate(i) \div 3 \quad (9)$$

(i = 47,20,or 14)

Response Time

The sum of the average training time and testing time of three feature sets is used as the response time as shown in (10).

$$Average\ of\ Response\ Time = \Sigma(\frac{TrainingTime\ per\ record(i)}{3} + \frac{TestingTime\ per\ record(i)}{3}) \quad (10)$$

$$(i = 47, 20, \text{or } 14)$$

4.7.2 Research Question 1 and Hypothesis 1

Q1: Which network traffic features are most effective for classifying benign traffic from IoT malware attacks?

H1: A set of features in the given datasets are the most indicative of classifying IoT malware attacks from normal traffic.

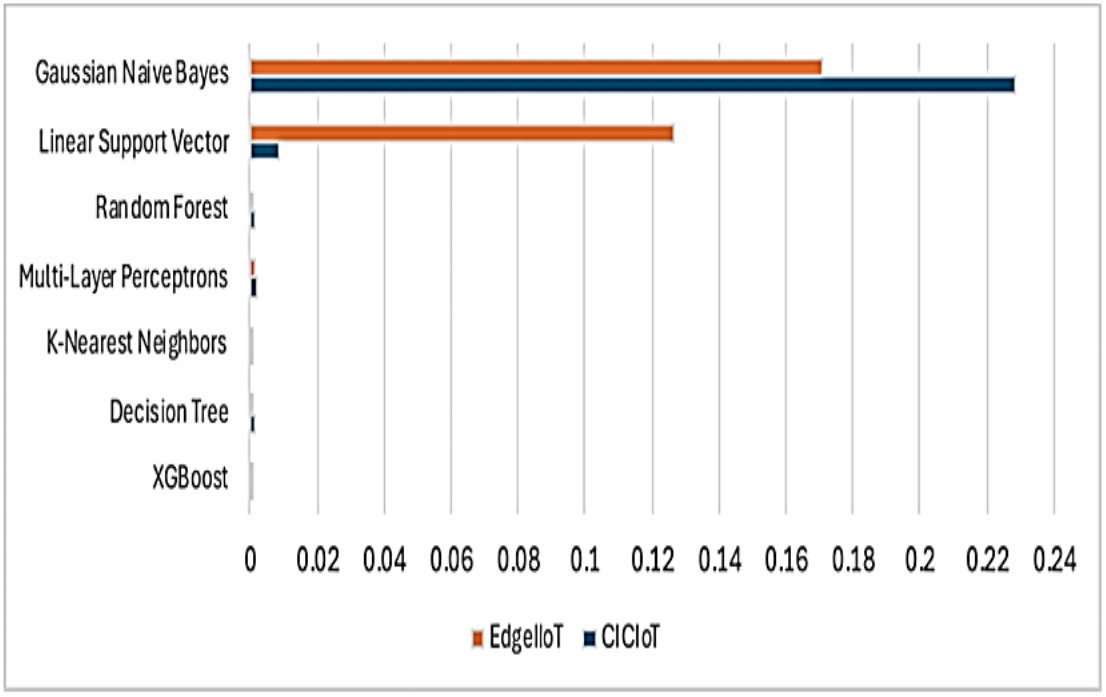


Figure 4.8 Binary Classification Accuracy Standard Deviation

In Figure 4-8, we are seeing the standard deviation of accuracy on a variety of feature test sets. Out of all classifiers, XGBoost has the best consistency value at 0.000802291. Gaussian Naïve Bayes (GNB) on the other hand has the greatest standard deviation of 0.228570281.

For EdgeIIoT test cases the accuracy standard deviations are consistent across three test cases. As before, XGBoost's performance is the lowest standard deviation (0.00004239), which suggests that the hypothesis is correct that these 14 features very strongly indicate intrusion attacks.

4.7.3 Research Question 2 and Hypothesis 2

Q2: Which machine learning model demonstrates the highest detection accuracy and overall performance while maintaining low false positive rates?

H2: XGBoost demonstrates the highest overall classification performance among the seven evaluated machine learning models.

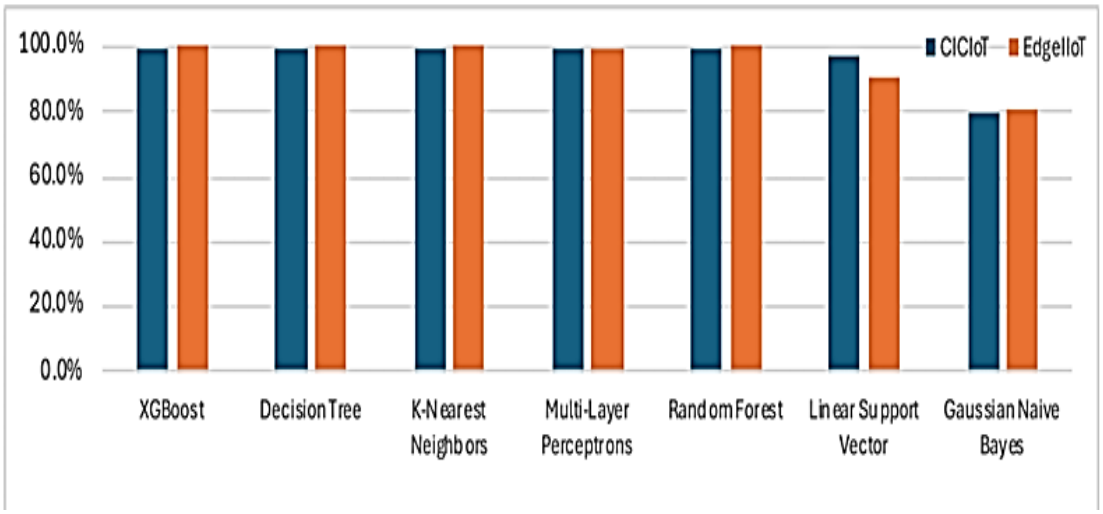


Figure 4.9 Binary Classification Average of Accuracy

The accuracy scores of average accuracy of seven machine learning models on the CICIoT dataset are shown in figure 4-9. Six models had an accuracy of more than 95%, excluding the GNB. The average accuracy for GNB is a lot lower at 79.645%. This difference is due to the effect of highly correlated features on its performance. Note that Naïve Bayes makes feature independence assumptions (Rish, 2011). The average accuracy of 7 classifiers is shown in the EdgeIIoT dataset. Five of these classifiers were able to exceed a 96% accuracy rate of average accuracy. Linear Support Vector Machine (LSVM) and GNB were not as good as others, achieving average accuracy of 91.003% and 81.307%, respectively.

4.7.4 Research Question 3 and Hypothesis 3

Q3. How can an end-to-end prototype integrating Machine Learning and Blockchain support intrusion detection and automated attack mitigation in IoT environments?

H3. Integrating the selected XGBoost model with Blockchain and smart contracts supports automated intrusion detection and mitigation by removing compromised IoT devices from the trusted network.

Table 4-5 Average Accuracy, Precision, Recall & F1Score (CICIoT)

CLASSIFIER	AVERAGE OF ACCURACY	AVERAGE OF RECALL	AVERAGE OF PRECISION	AVERAGE OF F1SCORE
XGBOOST	99.5%	99.6%	100.0%	99.8%
DECISION TREE	99.4%	99.4%	100.0%	99.7%
MULTI-LAYER PERCEPTRONS	99.0%	99.0%	100.0%	99.5%
K-NEAREST NEIGHBORS	99.0%	99.1%	99.9%	99.5%
RANDOM FOREST	98.9%	98.9%	100.0%	99.5%
LINEAR SUPPORT VECTOR	97.2%	97.3%	99.8%	98.5%
GAUSSIAN NAIVE BAYES	79.6%	79.2%	100.0%	87.0%

Table 4-6 Average Accuracy, Precision, Recall & F1Score (EdgeIIoT)

CLASSIFIER	AVERAGE OF ACCURACY	AVERAGE OF PRECISION	AVERAGE OF RECALL	AVERAGE OF F1SCORE
XGBOOST	99.995%	99.997%	99.997%	99.997%

DECISION TREE	99.945%	99.981%	99.949%	99.965%
RANDOM FOREST	99.946%	99.978%	99.954%	99.966%
K-NEAREST NEIGHBORS	99.972%	99.985%	99.979%	99.982%
MULTI-LAYER PERCEPTRONS	99.892%	99.975%	99.888%	99.931%
LINEAR SUPPORT VECTOR	91.003%	97.778%	90.399%	93.720%
GAUSSIAN NAIVE BAYES	81.307%	99.506%	76.587%	85.460%

As shown in Tables 4-5 and 4-6, XGBoost achieved the highest overall classification performance among the evaluated models in terms of accuracy, precision, recall, and F1-score. For the CICIoT dataset, XGBoost achieved an average accuracy of 99.55%, recall of 99.576%, precision of 99.962%, and F1-score of 99.769%. For the EdgeIIoT dataset, XGBoost achieved the highest accuracy (99.995%), precision (99.997%), recall (99.997%), and F1-score (99.997%). These results support Hypothesis 2 (H2), which states that XGBoost demonstrates the highest overall classification performance among the seven evaluated machine learning models.

Table 4-7 CICIoT Dataset Binary Classification Response Time

CLASSIFIER	**TRAINING SPEED PER RECORD (MICROSECONDS)**	**TESTING SPEED PER RECORD (MICROSECONDS)**	**TOTAL PROCESSING TIME PER RECORD(MICROSECONDS)**	**RESPONSE TIME RANKING**
GAUSSIAN NAIVE BAYES	0.31	0.15	0.46	1
XGBOOST	1.85	0.24	2.09	2

DECISION TREE	11.90	0.10	12.01	3
RANDOM FOREST	61.11	0.81	61.92	4
MULTI-LAYER PERCEPTRONS	324.42	0.55	324.97	5
LINEAR SUPPORT VECTOR	1904.54	0.06	1904.60	6
K-NEAREST NEIGHBORS	1.21	135490.79	135492.00	7

Table 4-8 EdgeIIoT Dataset Binary Classification Response Time

CLASSIFIER	TRAINING SPEED PER RECORD (MICROSECONDS)	TESTING SPEED PER RECORD (MICROSECONDS)	TOTAL PROCESSING TIME PER RECORD(MICROSECONDS)	RESPONSE TIME RANKING
GAUSSIAN NAIVE BAYES	0.33	0.27	0.60	1
XGBOOST	1.97	0.28	2.25	2
DECISION TREE	5.51	0.06	5.57	3
RANDOM FOREST	57.49	0.93	136.10	4
MULTI-LAYER	136.10	0.54	394.98	5

PERCEPTRONS				
LINEAR SUPPORT VECTOR	394.92	0.06	394.98	6
K-NEAREST NEIGHBORS	0.74	57992.50	57993.24	7

The results presented in Tables 4-7 and 4-8 show that GNB has the shortest response time in both datasets, while XGBoost ranks second in both binary classification experiments. Although GNB provides faster processing, XGBoost achieves substantially stronger classification performance. Therefore, XGBoost provides a practical balance between classification performance and processing time for the proposed intrusion detection component.

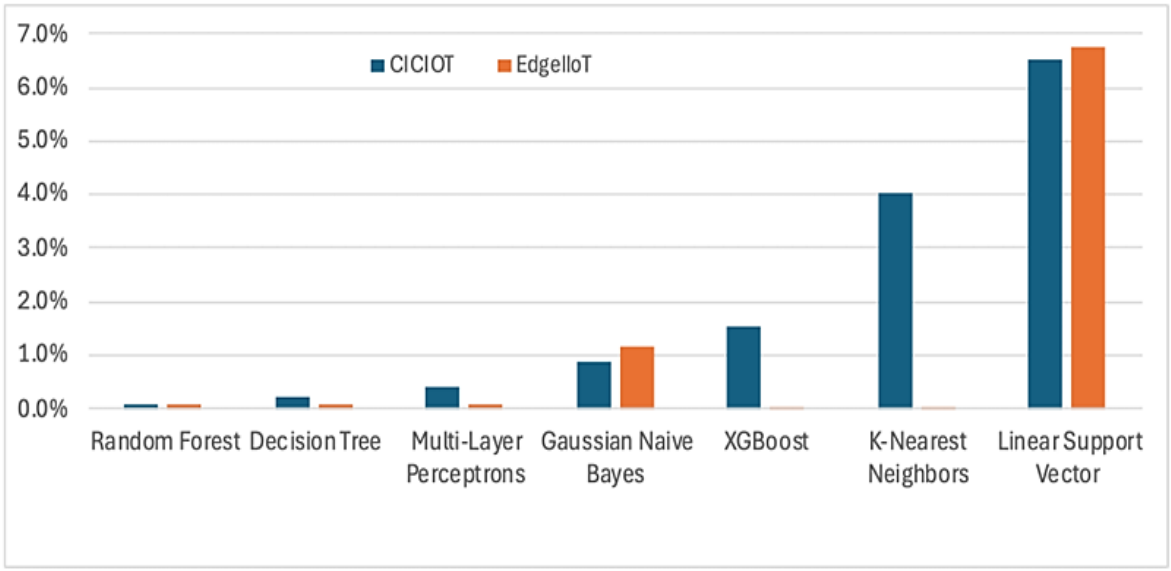


Figure 4.10 Binary Classification False Positive Rate

False Positive Rate Analysis

The false positive rate (FPR) results differ between the CICIoT2023 and EdgeIIoT datasets, as shown in Figure 4-10. For the CICIoT2023 dataset, Random Forest achieved the lowest FPR

at 0.079%, while XGBoost had a higher FPR than Random Forest, Decision Tree, and Multi-Layer Perceptron. In contrast, for the EdgeIIoT dataset, XGBoost achieved the lowest FPR at 0.012% among the seven evaluated models. The variation in FPR across the two datasets may be related to differences in dataset characteristics, class distributions, or sampling procedures. Therefore, FPR results are interpreted according to the specific dataset and classification setting.

4.8 Multiclass Classification

Multiclass classification was performed to identify different categories of malicious and benign IoT network traffic. The seven machine learning models were evaluated using the CICIoT2023 and EdgeIIoT datasets. Unlike binary classification, which separates traffic into benign and malicious classes, multiclass classification distinguishes among multiple traffic and attack categories.

The performance of the models was evaluated using accuracy, precision, recall, and F1-score. The results obtained from the two datasets are presented in the following tables and figures to compare the classification performance of the evaluated models across different traffic and attack categories.

Table 4-9 Multiclass Attack Category and Encoding

CICIoT Categorical Encoding	**CICIoT Attack Category**	**EdgeIIoT Classes**	**EdgeIIoT Attack Category**	**CICIoT Categorical Encoding**
0	Benign	0	Benign	0
1	Brute Force	1	DoS/DDoS	1
2	DDoS	2	Man-in-the-middle	2
3	DoS	3	Ransomware	3
4	Mirai	4	Reconnaissance	4
5	Reconnaissance	5	Web-based	5
6	Spoofing			6
7	Web-based			7

4.8.1 Multiclass Classification Measurement Metrics

The macro-average is a metric computed on a per-class basis, then equally averaged regardless of the number of classes. In contrast, the micro-average is derived from the sum of all classes' averages, resulting in an overall average metric. Micro-accuracy can be particularly useful when dealing with imbalanced data. In this research, we opt for metrics calculated using the micro-average due to the imbalanced nature of the CICIoT and EdgeIIoT datasets.

Macro Accuracy

The macro accuracy in (11) is calculated by averaging the accuracies among all classes by dividing the number of classes.

$$Macro\ Accuracy = \frac{(\Sigma(Accuracy\ (all\ classes)))}{Number\ of\ Classes} \qquad (11)$$

Micro Accuracy

Micro accuracy is calculated as the proportion of correctly classified instances (true positives) over the number of instances (true positives and false positives) for each class. Micro accuracy is beneficial in the context of imbalanced datasets like the selected CICIoT and EdgeIIoT datasets. In imbalanced data sets, there can be a large difference between the number of instances in different classes. Using a macro measure that gives equal weight to all classes will give a majority of classes the upper hand. The micro accuracy, on the other hand, gives a balanced picture of the accuracy of the classification process for all the classes. It makes sure that no particular class has undue effect on the overall accuracy.

$$Micro\ Accuracy = \frac{(\Sigma(TP\ (all\ classes)) + \Sigma\ (TN\ (all\ classes)))}{(\Sigma(TP(all\ classes) + \Sigma(TN(all\ classes)) + \Sigma(FP(all\ class) + \Sigma(FN(all\ classes)))} \qquad (12)$$

Feature Significance by Standard Deviation of Accuracy

The standard deviation of accuracy calculation in multiclass classification is the same as in the binary classification, as shown in (13). The accuracy test is based on three sets of features (47, 20, and 14), as explained in Section 4.4.3.2, Feature Selection. The standard deviation of accuracy is computed for 3 sets of testing scenarios to determine the most representative features that can be used to detect intrusion, i.e., features with the lowest standard deviation of accuracy are selected.

The standard deviation of accuracy is a measure of the variability of the accuracy that is expected from the mean accuracy. The smaller the standard deviation, the more all the accuracies are in the neighborhood of the mean.

$$\text{Standard Deivation of Accuracy} = \frac{\sqrt{\Sigma(\text{Accuracy}(i) - \text{Mean Accuracy})^2}}{3} \quad (13)$$

$$(\text{i} = 47, 20, 14)$$

Micro Precision / Micro Recall / F1 Score

First, we compute the total amount of True Positive (TP), False Positive (FP), and False Negative (FN) predictions for all classes. Precision is defined as the number of true positives / (true positives + false positives). Recall is the number of the total true positives divided by the number of total true positives plus the number of false negatives. According to this formula, the micro precision value is equal to the micro recall value, the micro F1 score, and the micro accuracy (Grandini et al., 2020).

False Positive Rate

The definition of the false positive rate for multiclass classification is similar to that of binary classification; it is expressed as FP/(FP+TN). In the multiclass case, however, FP is the total of the false positive counts for all classes, and TN is the total of the true negative counts for all classes.

Response Time

The response time in multiclass classification (14) is calculated as the sum of the average training times and the average testing times of three feature sets, similar to binary classification. binary classification.

$$Average\ of\ Response\ Time = \Sigma\left(\frac{TrainingTime\ per\ record(i)}{3} + \frac{TestingTime\ per\ record(i)}{3}\right) \quad (14)$$

4.7.2.2 Research Question 1 and Hypothesis 1

Q1: What are the most relevant features of the datasets to classify IoT malware attacks from normal traffic?

H1: In the provided datasets, a set of features are the most representative to classify IoT malware attacks from normal traffic.

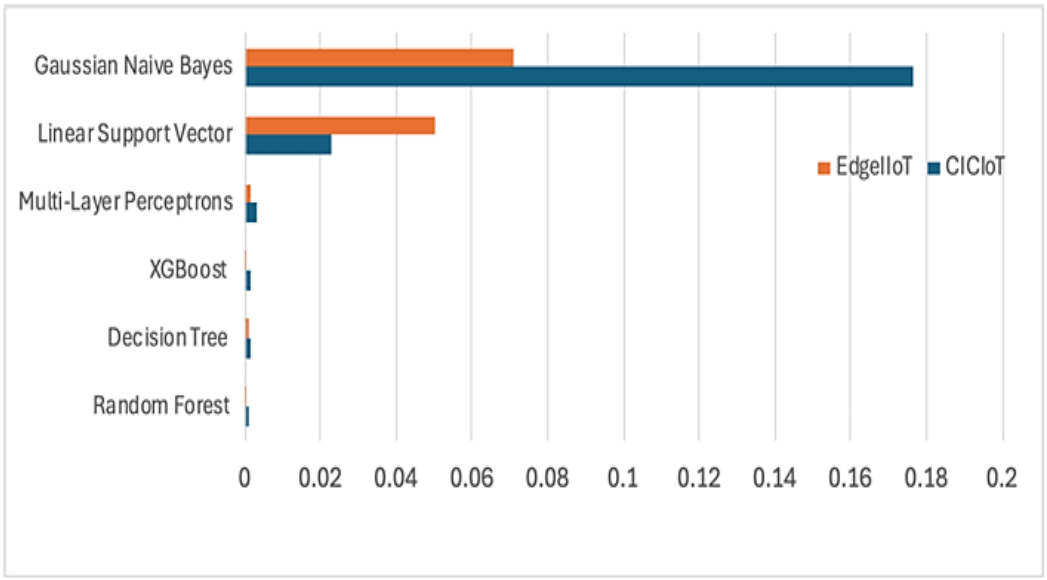


Figure 4.11 Multiclass Classification Accuracy Standard Deviation

The micro-accuracy standard deviation of three test scenarios with different feature sets is shown in Figure 4-11. The value of five classifiers is less than 0.1, which means that the classifiers are clearly not random. A significant difference between the GNB classifier and its classifiers can be explained by the assumption of independent features (Rish, 2011). However, due to the process stopping early in the process, the KNN result does not exist for all test scenarios. In a positive direction, the results obtained from EdgeIIoT show encouraging results, with all the standard deviations being under 0.1. Moreover, the means of the multiclass test results of the two datasets are similar, which also confirms the results of the binary tests; this shows that a little subset of features is highly indicative of classifying an intrusion attack.

4.7.2.3 Research Question 2 and Hypothesis 2

Q2: Is it possible to get a detection accuracy rate higher than 95% for machine learning models in detecting IoT malware attacks in the proposed datasets?

H2: The multiclass classification results show that the strongest models achieved high classification performance, while GNB and LSVM showed comparatively lower performance. XGBoost demonstrated the strongest overall performance among the evaluated models.

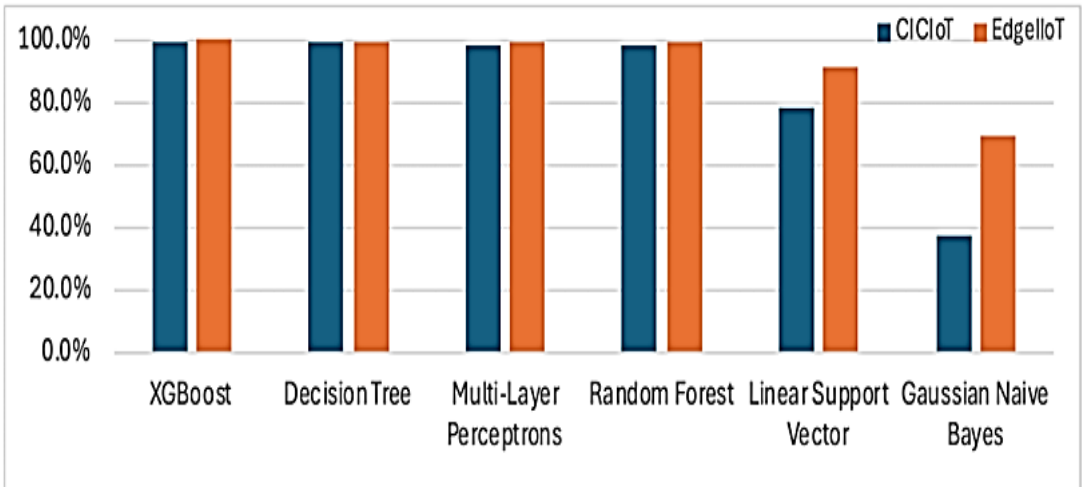


Figure 4.12 Multiclass Classification Average of Micro Accuracy

Multiclass classification results are very similar to binary classification results, as demonstrated in Figures 4-12. The micro accuracy is on average higher than 95% with four models.

The results of LSVM and GNB are still below this threshold, as in the binary classification results. After running for more than a week, we had to stop the KNN classifier because of its slow performance.

4.7.2.4 Research Question 3 and Hypothesis 3

Q3. How can an end-to-end prototype integrating Machine Learning and Blockchain support intrusion detection and automated attack mitigation in IoT environments?

H3. Integrating the selected XGBoost model with Blockchain and smart contracts supports automated intrusion detection and mitigation by removing compromised IoT devices from the trusted network.

In multiclass classification, micro accuracy is the same as micro precision, micro recall and micro F1-score, as explained in Section 4.7.2.1. Based on the results displayed in Figures 4–10, XGBoost is the most effective model of all the models tested.

Table 4-10 CICIoT Dataset Multiclass Classification Response Time

CLASSIFIE	TRAINING SPEED PER RECORD (MICROSECONDS)	TESTING SPEED PER RECORD (MICROSECONDS)	TOTAL PROCESSING TIME PER RECORD (MICROSECONDS)	RESPONSE TIME RANKING

GAUSSIAN NAIVE BAYES	0.31732725	0.70078350	1.01811075	1
XGBOOST	11.06997221	1.43960816	12.50958037	2
DECISION TREE	14.65697825	0.05511122	14.71208948	3
RANDOM FOREST	124.58281828	1.19768160	125.78049988	4
LINEAR SUPPORT VECTOR	11314.62434774	0.14619485	11314.77054259	5
MULTI-LAYER PERCEPTRONS	1907.67787131	108.60322260	2016.28109391	6

Table 4-11 EdgeIIoT Dataset Multiclass Classification Response Time

CLASSIFIER	TRAINING SPEED PER RECORD (MICROSECONDS)	TESTING SPEED PER RECORD (MICROSECONDS)	TOTAL PROCESSING TIME PER RECORD (MICROSECONDS)	RESPONSE TIME RANKING
GAUSSIAN NAIVE BAYES	0.24751046	0.26948669	0.51699715	1
DECISION TREE	7.37769756	0.08091129	7.45860886	2
XGBOOST	9.72397180	1.14389175	10.86786355	3
RANDOM FOREST	28.67503374	1.24150696	29.91654070	4

MULTI-LAYER PERCEPTRONS	882.08587524	0.57680088	882.66267612	5
LINEAR SUPPORT VECTOR	4158.08783930	0.07942904	4158.16726834	6

Tables 4-10 and 4-11 present the training speed, testing speed, total processing time per record, and response-time ranking of the six classifiers with available multiclass response-time results. K-Nearest Neighbors (KNN) is not included in these tables because its execution was stopped due to excessive processing time.

For the CICIoT2023 dataset, Gaussian Naive Bayes (GNB) achieved the shortest total processing time per record, followed by XGBoost. Decision Tree and Random Forest ranked third and fourth, respectively. Among the models reported in Table 4-10, Multi-Layer Perceptron required the longest total processing time.

For the EdgeIIoT dataset, GNB again achieved the shortest total processing time per record, followed by Decision Tree and XGBoost. Random Forest, Multi-Layer Perceptron, and Linear Support Vector Machine followed in the reported ranking. These results show that GNB provides the fastest processing among the reported models, whereas XGBoost offers stronger overall classification performance. Therefore, model selection should consider both classification performance and processing time.

In the CICIoT dataset, the fastest is GNB with a processing time of 0.00000102 seconds per record followed by XGBoost with 0.00001251 seconds per record. DT and RF are in 3rd position and 4th position. The processing times of KNN, LSVM and MLP are much higher, the most time consuming being MLP.

In the EdgeIIoT dataset, GNB has once again the fastest processing time, which is 0.00000052 seconds per record. DT is the second-best algorithm, followed by XGBoost. RF comes in fourth, followed by MLP and LSVM. KNN is the slowest with the processing time of 0.08716042 seconds per record. GNB is the fastest, but its accuracy is not good enough, so XGBoost is a better option for IoT intrusion detection.

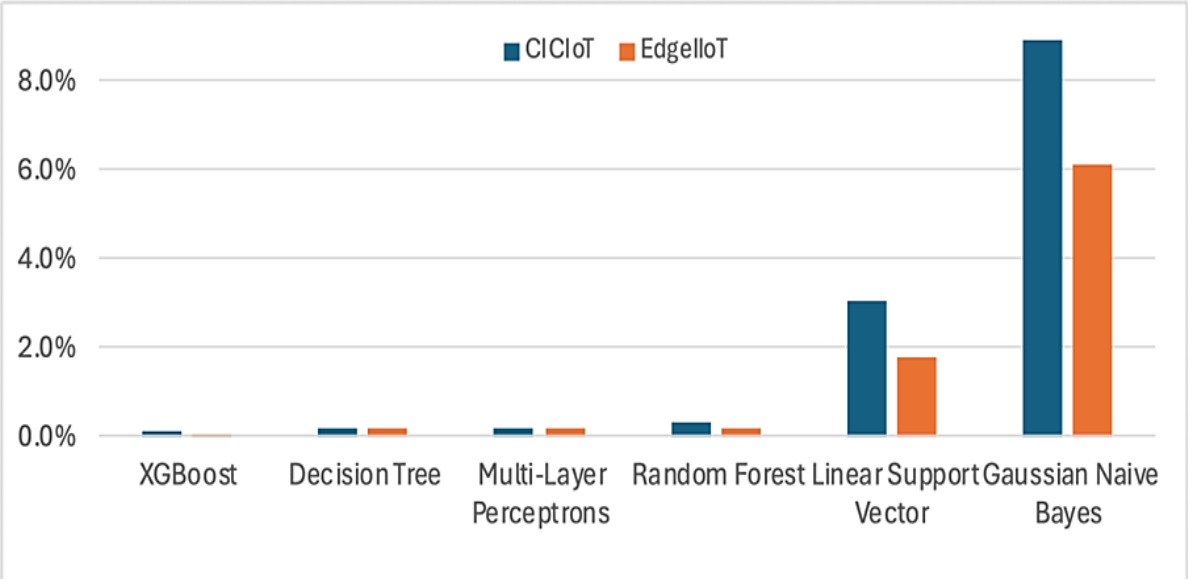


Figure 4.13 Multiclass Classification False Positive Rate

Figure 4-13 presents the false positive rate (FPR) results for multiclass classification on the CICIoT2023 and EdgeIIoT datasets. The FPR results should be interpreted separately for each dataset because model performance may vary across classification settings. A lower FPR indicates fewer false positive predictions, while a higher FPR indicates more false alarms. These results should be considered alongside classification performance when selecting a model for IoT intrusion detection.

4.8 Blockchain-Based Detection and Mitigation Simulation

The end-to-end prototype was developed to demonstrate the integration of Machine Learning and Blockchain for intrusion detection and automated attack mitigation in an IoT environment. The prototype uses the selected XGBoost model for intrusion detection and a permissioned Ethereum blockchain with smart contracts for managing compromised devices. The simulation demonstrates the complete process from attack traffic generation and detection to automated mitigation.

4.8.1 End-to-End Prototype Implementation

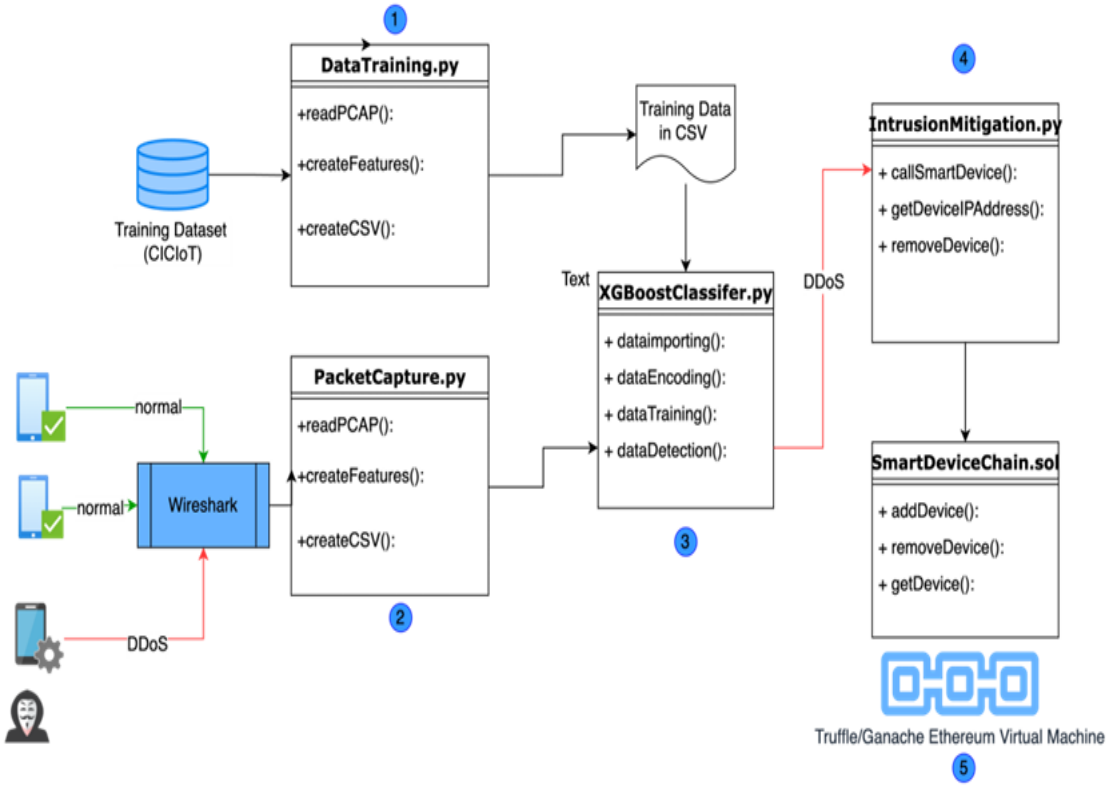


Figure 4.14 Blockchain Enabled Machine Learning IoT Framework in UML

Figure 4-14 presents the proposed Machine Learning and Blockchain-based IoT security framework using a Unified Modeling Language (UML) diagram. The prototype provides a proof-of-concept implementation that integrates Python-based machine learning with smart contracts developed in Solidity. The smart contracts are deployed on the Ethereum Virtual Machine (EVM) using Ganache as the local blockchain environment, while Truffle is used for smart-contract development and deployment.

using Unified Modeling Language (UML) diagram. In this place, we offer a proof-of-concept implementation, including Python programs and smart contracts written in the Solidity programming language, which is used to build on the Ethereum Virtual Machine (EVM). Ganache™ is the EVM running on the local machine, and serves as a local blockchain for testing.

Truffle™ is the blockchain development tool to deploy smart contracts to this local blockchain network. The prototype includes 5 embedded components in a blockchain smart contract platform:

Data training on the XGBoost model: The first part is to train the XGBoost classifier on the CICIoT dataset. The decision on choosing the XGBoost classifer is based on its high accuracy, false positive rate and response time.

2. Real-time packet simulation and capture: Cooja produces benign network packets.

Meanwhile, an Attack Packer script is generating DDoS attack packets in Python. All the network packets are captured in Wireshark and these are converted into PCAP data files, where the features are identified using the results of Hypothesis 1 and converted to the CSV format.

3. Intrusion Detection: The collected data is stored in a CSV file and is then used in the trained XGBoost Classifier to detect intrusions.

Misbehaviour is detected and a list of all devices involved in the misbehaviour is passed to the smart contract deployed in the local Ganache EVM along with the address of the infected device.

5. Device removal: The smart contract will take the device out of the network and will send this information to all the nodes of the network.

The purpose of this framework is to illustrate how to apply the combination of machine learning and blockchain technology to improve the security of the IoT network, particularly to identify and prevent the spread of malicious attacks.

4.8.3 Working Mechanism of the Proposed Model

The functionality of the proposed block-chains enabled machine learning (BEML) framework is explained in the following steps in detail:

A new Ethereum Virtual Machine (EVM) workspace is created within the local Ganache environment and three smart contracts are deployed: Migrations, StringToAddress and SmartDeviceChain. A smart contract is really just a program, with functions and data that live on the Ethereum blockchain at a particular address on the blockchain (Figure 4-15).

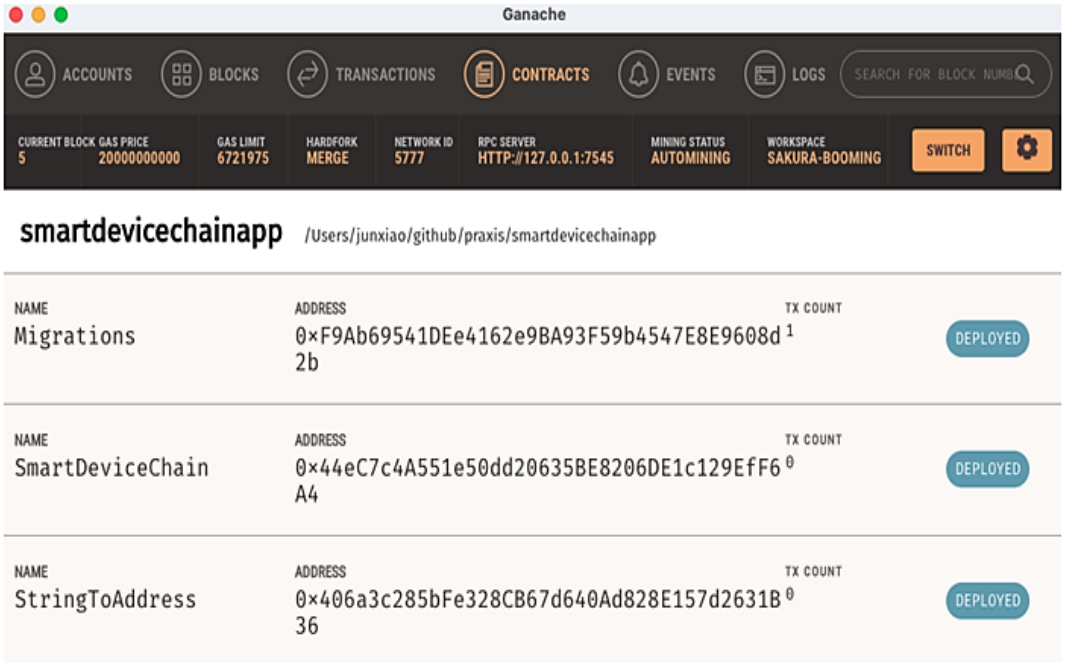


Figure 4.15 Smart Contracts Deployed in the Ethereum Virtual Machine

2. **Registration of New Devices:** Five devices are registered to one of the accounts on the blockchain by executing SmartDeviceChain. addDevice(). This marks the initial step in registering trusted devices on the blockchain. Following successful registration, the device's credential information is securely stored on the blockchain, ensuring both immutability and transparency (see Figure 4-16).

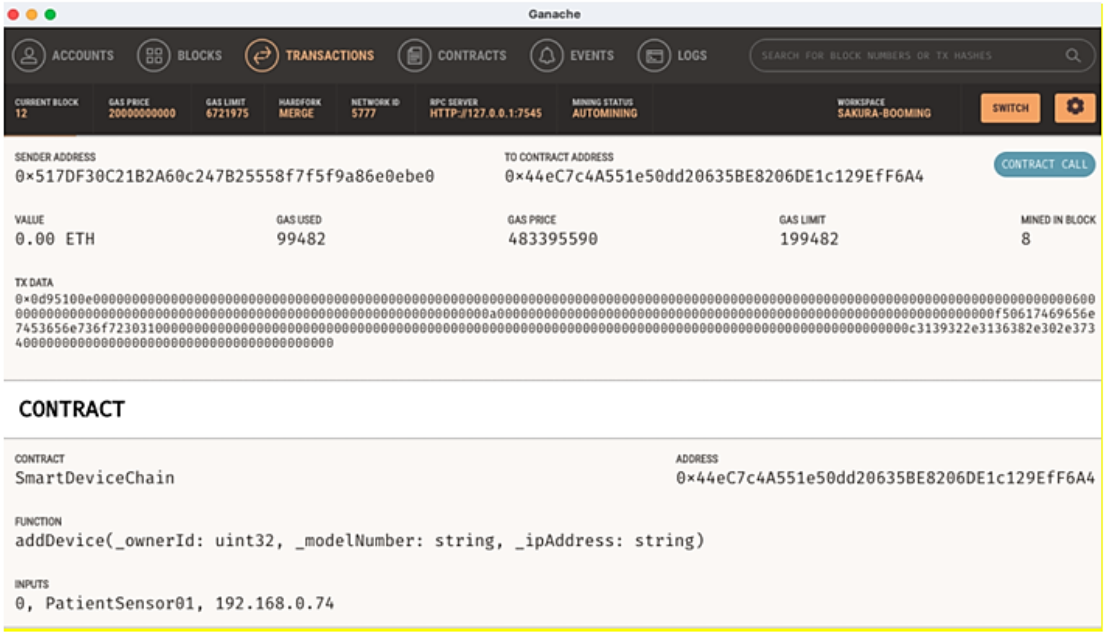


Figure 4.16 Register New Devices in the Ethereum Virtual Machine

The 3rd step is to generate live network traffic and generate a DDoS attack using a Python script, which sends 10 DDoS SYN flood packets to the device with IP address 192.168.0.72, from which the benign traffic is simulated using Cooja. The packets are stored in Wireshark and then exported from the PCAP to a 14-feature CSV file.

Step 4: Intrusion Detection: In this step, the captured traffic is passed to the trained XGBoost Classifier for Intrusion Detection to determine whether the packet is a DDoS attack or not. If the attack flag is TRUE, the attacked device's IP address will be captured for further actions.

5. Intrusion Mitigation: The infected device's IP address is passed to SmartDeviceChain. Then the smart contract can call removeDevice() so that it can deactivate the malicious device and send the news to the entire blockchain (Figure 4-17).

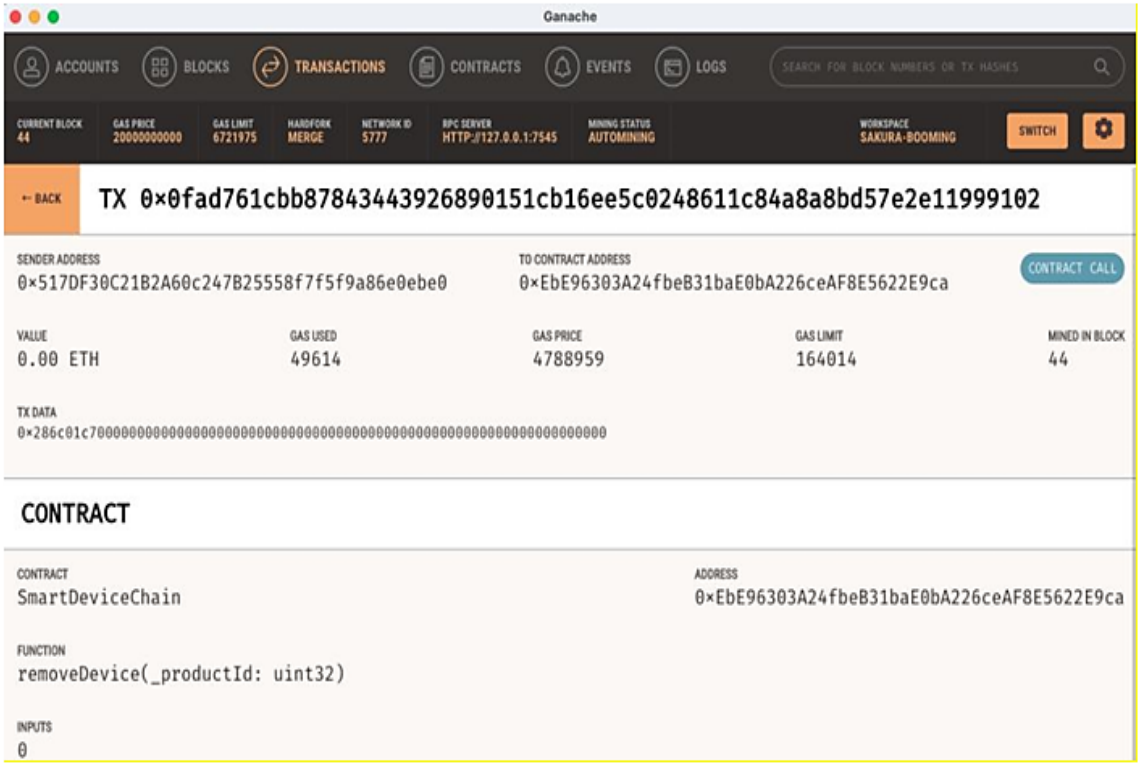


Figure 4.17 Remove Compromised Device in the Ethereum Virtual Machine

4.9 Summary

This chapter presented the experimental results of the proposed Machine Learning and Blockchain-based intrusion detection framework for IoT environments. The CICIoT2023 and EdgeIIoT datasets were evaluated using seven machine learning models: Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP).

The results included exploratory data analysis, data preprocessing, feature selection, hyperparameter tuning, and binary and multiclass classification. The machine learning models were evaluated using accuracy, precision, recall, F1-score, response time, and false positive rate. The feature-selection analysis identified a final set of 14 relevant network traffic features for the classification experiments.

The comparative results showed that XGBoost provided the strongest overall classification performance among the evaluated models. However, the experimental results did not support the statement that all seven models consistently achieved an accuracy exceeding 95%. Response-time and false-positive-rate results also varied across the models and datasets.

Finally, an end-to-end prototype was implemented to demonstrate the integration of the selected XGBoost model with a permissioned Ethereum blockchain and smart contracts. The prototype demonstrated the detection of malicious traffic and automated mitigation through the removal or revocation of compromised IoT devices from the trusted device registry.

Overall, the experimental results demonstrate the potential of integrating Machine Learning and Blockchain technologies for intelligent intrusion detection and automated attack mitigation in IoT environments.

Table 4-12 Summary of Results and Validated Hypotheses

#	Hypothesis	Result (CICIoT)	Result (EdgeIIoT)	Hypothesis Validated
1	H1: Feature selection	The combined feature-selection approach identified 14 relevant features for the classification experiments.	The combined feature-selection approach identified 14 relevant features for the classification experiments.	Yes, based on the reported feature-selection findings.
2	H2: XGBoost performance	XGBoost achieved 99.5% binary classification accuracy.	XGBoost achieved 99.995% binary classification accuracy.	Supported by the reported overall classification results.
3	H3: ML and Blockchain integration	The prototype demonstrated malicious-traffic detection and mitigation through the removal or revocation of a compromised device.	The prototype demonstrated malicious-traffic detection and mitigation through the removal or revocation of a compromised device.	Supported by the prototype demonstration.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

This research investigated the development of an intelligent and decentralized intrusion detection framework for IoT environments by integrating Machine Learning and Blockchain technologies. Seven machine learning models, namely Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP), were evaluated using the CICIoT2023 and EdgeIIoT datasets. The evaluation considered binary and multiclass classification using accuracy, precision, recall, F1-score, response time, and false positive rate. The feature-selection analysis addressed the first research question by evaluating different subsets of network traffic features. The study compared 47, 20, and 14 feature sets using multiple feature-selection techniques, including Pearson Correlation, Variance Threshold, Mutual Information, and Chi-Square Test. The combined feature-selection approach identified 14 relevant features for the final classification experiments. The results indicate that the selected 14 features provide an effective and consistent representation of the network traffic for intrusion detection.

The second research question focused on identifying the machine learning model with the strongest overall classification performance while maintaining acceptable false positive rates. The experimental results show that XGBoost achieved the strongest overall classification performance among the seven evaluated models. In the binary classification experiments, XGBoost achieved 99.5% accuracy on CICIoT2023 and 99.995% accuracy on EdgeIIoT. The multiclass results also showed strong performance by XGBoost compared with the other evaluated models. These findings support Hypothesis 2 (H2), which states that XGBoost demonstrates the highest overall classification performance among the seven evaluated machine learning models.

The response-time results show that GNB achieved the shortest processing time among the evaluated models. Therefore, although XGBoost provided the strongest classification performance, GNB was faster in terms of response time. This indicates that model selection for practical IoT intrusion detection may require a balance between classification performance and processing time.

The false positive rate results varied across the datasets and classification tasks. For binary classification on CICIoT2023, Random Forest achieved the lowest false positive rate, while XGBoost achieved the lowest false positive rate on EdgeIIoT. In the multiclass classification results, XGBoost demonstrated strong false positive rate performance. Therefore, the results

should be interpreted according to the specific dataset and classification setting rather than assuming that one model has the lowest false positive rate in every experiment.

The third research question examined how an end-to-end prototype integrating Machine Learning and Blockchain can support automated intrusion detection and attack mitigation. The proposed prototype integrates the selected XGBoost classifier with a permissioned Ethereum blockchain and smart contracts. Network traffic is captured and converted into the required feature format, after which the trained classifier identifies malicious traffic. When a compromised device is detected, its information is transferred to the blockchain layer, where the smart contract performs the predefined mitigation action by removing or revoking the compromised device from the trusted device registry.

The prototype therefore demonstrates the practical integration of intelligent intrusion detection with decentralized security management and automated mitigation. These results support Hypothesis 3 (H3), which states that integrating the selected XGBoost model with Blockchain and smart contracts supports automated intrusion detection and mitigation by removing compromised IoT devices from the trusted network.

Overall, the findings demonstrate that combining effective feature selection, machine learning-based intrusion detection, and blockchain-based automated mitigation can provide an integrated approach to improving IoT security. The experimental results support the proposed research objectives and demonstrate the feasibility of the developed end-to-end prototype.

5.2 Conclusions

This thesis investigated an end-to-end security framework that integrates Machine Learning and Blockchain technologies for intrusion detection and automated attack mitigation in IoT environments. Seven machine learning models, including Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP), were evaluated using the CICIoT2023 and EdgeIIoT datasets for binary and multiclass classification.

The feature-selection analysis evaluated 47, 20, and 14 feature sets using Pearson Correlation, Variance Threshold, Mutual Information, and Chi-Square Test. The combined feature-selection approach identified 14 relevant network traffic features for the final classification experiments. The results demonstrate that the selected features provide an effective basis for distinguishing benign and malicious IoT traffic.

The experimental results show that XGBoost demonstrated the strongest overall classification performance among the seven evaluated machine learning models. In contrast, GNB achieved

the shortest response time. The false positive rate varied across datasets and classification settings, indicating that model selection should consider multiple performance measures rather than accuracy alone.

Based on the comparative evaluation, XGBoost was selected for integration into the proposed end-to-end security prototype. The prototype integrates the selected XGBoost model with a permissioned Ethereum blockchain and smart contracts. Cooja was used for IoT network simulation, while Wireshark was used for network traffic capture and analysis. The detected malicious device information was transferred to the blockchain layer, where smart contracts supported automated mitigation by removing or revoking the compromised device from the trusted device registry.

Overall, the study demonstrates the feasibility of combining Machine Learning and Blockchain to provide intelligent intrusion detection, decentralized security management, and automated attack mitigation in IoT environments. The findings address the research objectives and support the proposed approach as a practical proof-of-concept for improving the security of IoT networks.

5.3 Contributions to Body of Knowledge

This study makes the following contributions to the field of IoT security:

1. Comparative Evaluation of Machine Learning Models:

This study makes the following contributions to the field of IoT security:

1. Comparative Evaluation of Machine Learning Models:

Seven machine learning models, including Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), Linear Support Vector Machine (LSVM), K-Nearest Neighbors (KNN), XGBoost, and Multi-Layer Perceptron (MLP), were comparatively evaluated for IoT intrusion detection using the CICIoT2023 and EdgeIIoT datasets.

2. Identification of Relevant Network Traffic Features:

A feature-selection process was applied using Pearson Correlation, Variance Threshold, Mutual Information, and Chi-Square Test. The combined approach identified a final set of 14 relevant network traffic features from the initial feature set for intrusion classification.

3. Comprehensive Performance Evaluation:

The machine learning models were evaluated for binary and multiclass classification using accuracy, precision, recall, F1-score, response time, and false positive rate. This provides a comprehensive comparison of model performance for IoT intrusion detection.

4. Integration of Machine Learning and Blockchain:

An end-to-end prototype was developed by integrating the selected XGBoost model with a permissioned Ethereum blockchain and smart contracts. The prototype combines intelligent intrusion detection with decentralized security management.

5. Automated Attack Mitigation:

The proposed prototype demonstrates automated mitigation of detected malicious devices through smart contracts. When a compromised device is identified, the corresponding device can be removed or revoked from the trusted device registry, supporting automated response to IoT security threats.

5.4 Recommendations for Future Research

This research demonstrates the feasibility of integrating Machine Learning with a permissioned Blockchain for intrusion detection and automated attack mitigation in IoT environments. Based on the findings and limitations of the study, the following recommendations are proposed for future research:

5.4.1 Evaluation Using Additional IoT Datasets:

Future studies should evaluate the proposed framework using additional IoT security datasets and real-world network traffic to further assess its generalizability and robustness.

5.4.2 Lightweight Machine Learning Models:

Future research should investigate lightweight machine learning models that can provide effective intrusion detection while requiring fewer computational resources, particularly for resource-constrained IoT devices.

5.4.3 Large-Scale IoT Testbed Evaluation:

The proposed framework should be evaluated in a larger-scale distributed IoT testbed to assess its performance and scalability under more realistic network conditions.

5.4.4 Blockchain Interoperability:

Future studies should investigate interoperability between different Blockchain platforms and IoT security infrastructures to support flexible deployment in heterogeneous IoT environments.

5.4.5 Adaptive and Online Learning:

Future research should explore adaptive feature-selection and online learning techniques to address changing IoT traffic patterns and improve the detection of emerging and previously unseen attacks.

REFERENCES

[1] R. Alajlan, N. Alhumam, and M. Frikha, "Cybersecurity for blockchain-based IoT systems: A review," *MDPI AG*, 2023, doi: 10.3390/app13137432.

[2] A. A. Alahmadi et al., "DDoS Attack Detection in IoT-Based Networks Using Machine Learning Models: A Survey and Research Directions," *Electronics*, vol. 12, p. 3103, 2023, doi: 10.3390/electronics12143103.

[3] A. S. Albulayhi and I. S. Alsukayti, "A Blockchain-Centric IoT Architecture for Effective Smart Contract-Based Management of IoT Data Communications," *Electronics*, vol. 12, p. 2564, 2023, doi: 10.3390/electronics12122564.

[4] M. V. Anand et al., "Gaussian naïve bayes algorithm: A reliable technique involved in the assortment of the segregation in cancer," *Hindawi Limited*, 2022, doi: 10.1155/2022/2436946.

[5] K. Ashton, "That 'Internet of Things' Thing: In the Real World Things Matter More than Ideas," *RFID Journal*, 2009.

[6] E. S. Babu et al., "Blockchain-based intrusion detection system of IoT urban data with device authentication against DDoS attacks," *Computers & Electrical Engineering*, vol. 103, p. 108287, 2022, doi: 10.1016/j.compeleceng.2022.108287.

[7] S. M. Baig et al., "A blockchain and stacked machine learning approach for malicious nodes' detection in internet of things," *Peer-to-peer Networking and Applications*, vol. 16, no. 6, pp. 2811-2832, 2023, doi: 10.1007/s12083-023-01554-1.

[8] Y. Zhang, Y. Sun, X. Wang, and Z. Zheng, "Blockchain-Based Trusted Data Sharing Among Intelligent IoT Devices," *IEEE Access*, vol. 7, pp. 162-173, 2019.

[9] Y. Wang, X. Li, and J. Zhang, "Blockchain-Enabled Secure Intrusion Detection for Internet of Things: A Survey," *Computer Networks*, 2024.

[10] Z. S. Mahdi, R. M. Zaki, and L. Alzubaidi, "A Secure and Adaptive Framework for Enhancing Intrusion Detection in IoT Networks Using Incremental Learning and Blockchain," *Security and Privacy*, vol. 8, p. e70071, 2025.

[11] S. Bhaskar and A. Purohit, "Amalgamation Ethereum smart contracts to routing information protocol to extenuate DDoS attacks," in *Proceedings of International Conference on Recent Trends in Computing*, Springer Nature Singapore, pp. 615-626, 2023, doi: 10.1007/978-981-19-8825-7_53.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.

[13] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794, 2016, doi: 10.1145/2939672.2939785.

[14] Y. Chen, J. Hou, Q. Li, and H. Long, "DDoS attack detection based on random forest," *IEEE*, 2020, doi: 10.1109/pic50277.2020.9350788.

[15] J. Cheng et al., "Cooperative detection method for DDoS attacks based on blockchain," *Computer Systems Science and Engineering*, vol. 43, no. 1, pp. 103-117, 2022, doi: 10.32604/csse.2022.025668.

[16] [16] B. D. Davis, J. C. Mason, and M. Anwar, "Vulnerability studies and security postures of IoT devices: A Smart Home Case Study," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9898-9908, 2020.

[17] DPKT, "Dpkt Documentation," 2022. [Online]. Available: https://dpkt.readthedocs.io/en/latest/

[18] J. Duque, "The IoT to Smart Cities - A design science research approach," *Procedia Computer Science*, vol. 219, pp. 279-285, 2023, doi: 10.1016/j.procs.2023.01.291.

[19] M. Essaid, D. Kim, S. H. Maeng, S. Park, and H. T. Ju, "A Collaborative DDoS Mitigation Solution Based on Ethereum Smart Contract and RNN-LSTM," in *20th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, Matsue, Japan, pp. 1-6, 2019, doi: 10.23919/APNOMS.2019.8892947.

[20] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning," *IEEE Access*, 2022, doi: 10.1109/access.2022.3165809.

[21] G. Gaobotse, E. Mbunge, J. Batani, and B. Muchemwa, "Non-invasive smart implants in healthcare: Redefining healthcare services delivery through sensors and emerging digital health technologies," *Elsevier BV*, 2022, doi: 10.1016/j.sintl.2022.100156.

[22] R. Gaur et al., "A machine-Learning-Blockchain-based authentication using smart contracts for an IoHT system," *MDPI AG*, 2022, doi: 10.3390/s22239074.

[23] G. Gkagkas, D. J. Vergados, A. Michalas, and M. Dossis, "The Advantage of the 5G Network for Enhancing the Internet of Things and the Evolution of the 6G Network," *Sensors*, vol. 24, no. 8, p. 2455, 2024, doi: 10.3390/s24082455.

[24] M. Grandini, E. Bagli, and G. Visani, "Metrics for Multi-Class Classification: an Overview," *ArXiv*, abs/2008.05756, 2020.

[25] H. Honar Pajoo, M. Rashid, F. Alam, and S. Demidenko, "Hyperledger fabric blockchain for securing the edge internet of things," *MDPI AG*, 2021, doi: 10.3390/s21020359.

[26] R. F. Ibrahim, Q. Abu Al-Haija, and A. Ahmad, "DDoS attack prevention for internet of thing devices using Ethereum blockchain technology," *Sensors*, vol. 22, no. 18, p. 6806, 2022, doi: 10.3390/s22186806.

[27] A. Iftikhar, K. N. Qureshi, M. Shiraz, and S. Albahli, "Security, trust and privacy risks, responses, and solutions for high-speed smart cities networks: A systematic literature review," *Elsevier BV*, 2023, doi: 10.1016/j.jksuci.2023.101788.

[28] International Institute for Management Development, "IMD Smart City Index Report 2023," 2023. [Online]. Available: https://imd.cld.bz/Smart-City-Index-20233

[29] W. Iqbal, H. Abbas, M. Daneshmand, B. Rauf, and Y. A. Bangash, "An in-depth analysis of IoT security requirements, challenges, and their countermeasures via software-defined security," *IEEE Internet of Things Journal*, 2020, doi: 10.1109/jiot.2020.2997651.

[30] M. Javaid, A. Haleem, R. P. Singh, R. Suman, and E. S. Gonzalez, "Understanding the adoption of industry 4.0 technologies in improving environmental sustainability," *Elsevier BV*, 2022, doi: 10.1016/j.susoc.2022.01.008.

[31] S. Jiang et al., "BSD-Guard: A collaborative blockchain-based approach for detection and mitigation of SDN-targeted ddos attacks," *Security and Communication Networks*, pp. 1-16, 2022, doi: 10.1155/2022/1608689.

[32] X. Jiang et al., "Credible link flooding attack detection and mitigation: A blockchain-based approach," *IEEE Transactions on Network and Service Management*, 2024, doi: 10.1109/tnsm.2024.3357660.

[33] R. Jmal et al., "Distributed Blockchain-SDN Secure IoT System Based on ANN to Mitigate DDoS Attacks," *Applied Sciences*, vol. 13, no. 8, p. 4953, 2023, doi: 10.3390/app13084953.

[34] A. I. Jony and A. K. B. Arnob, "A long short-term memory based approach for detecting cyber attacks in IoT using CIC-IoT2023 dataset," *Journal of Edge Computing*, 2024, doi: 10.55056/jec.648.

[35] I. Katib and M. Ragab, "Blockchain-assisted hybrid harris hawks optimization based deep DDoS attack detection in the IoT environment," *Mathematics*, vol. 11, no. 8, 2023, doi: 10.3390/math11081887.

[36] R. Khader and D. Eleyan, "Survey of DoS/DDoS attacks in IoT," *Research and Development Academy*, 2021, doi: 10.37868/sei.v3i1.124.

[37] M. Khalaf, A. Ayad, M. H. K. Tushar, M. Kassouf, and D. Kundur, "A survey on cyber-physical security of active distribution networks in smart grids," *IEEE Access*, 2024, doi: 10.1109/access.2024.3364362.

[38] H. Y. Kim, "Statistical notes for clinical researchers: assessing normal distribution (2) using skewness and kurtosis," *Restorative Dentistry & Endodontics*, vol. 38, no. 1, pp. 52-54, 2013, doi: 10.5395/rde.2013.38.1.52.

[39] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80-84, 2017, doi: 10.1109/mc.2017.201.

[40] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset," 2018.

[41] M. Kumar et al., "Healthcare Internet of Things (H-IoT): Current Trends, Future Prospects, Applications, Challenges, and Security Issues," *Electronics*, vol. 12, p. 2050, 2023, doi: 10.3390/electronics12092050.

[42] P. Kumar et al., "PPSF: A privacy-preserving and secure framework using blockchain-based machine-learning for IoT-driven smart cities," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 3, pp. 2326-2341, 2021, doi: 10.1109/TNSE.2021.3089435.

[43] H. Kurisaka, P. L. Nguyen, K. Nguyen, and H. Sekiya, "An evaluation of IoT device performance in Ethereum-Based IoT system," in *RIVF International Conference on Computing and Communication Technologies*, 2023, doi: 10.1109/rivf60135.2023.10471781.

[44] Y. Latief, "Sandworm unveiled as october 2022 ukraine infrastructure hackers," 2023. [Online]. Available: https://www.smart-energy.com/industry-sectors/cybersecurity/sandworm-unveiled-as-october-2022-ukraine-infrastructure-hackers/

[45] G. Lucky, F. Jjunju, and A. Marshall, "A lightweight decision-tree algorithm for detecting DDoS flooding attacks," *IEEE*, 2020, doi: 10.1109/qrs-c51114.2020.00072.

[46] MarketsandMarkets TM, "Industrial IoT market size, share and trends forecast to 2026," 2022. [Online]. Available: https://www.marketsandmarkets.com/Market-Reports/industrialinternet-of-things-market-129733727.html

[47] R. Mishra, B. K. R. Naik, R. D. Raut, and M. Kumar, "Internet of Things (IoT) adoption challenges in renewable energy: A case study from a developing economy," *Journal of Cleaner Production*, vol. 371, p. 133595, 2022, doi: 10.1016/j.jclepro.2022.133595.

[48] A. Munshi, A. Nouf, N. Alqarni, and A. Abdullah, "DDoS Attack on IOT Devices," in *3rd International Conference on Computer Applications & Information Security (ICCAIS)*, 2020, doi: 10.1109/ICCAIS48893.2020.9096818.

[49] S. Naiem, A. E. Khedr, A. M. Idrees, and M. I. Marie, "Enhancing the efficiency of gaussian naïve bayes machine learning classifier in the detection of DDOS in cloud computing," *IEEE Access*, 2023, doi: 10.1109/access.2023.3328951.

[50] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," *Bitcoin.org*, 2008. [Online]. Available: https://bitcoin.org/en/bitcoin-paper

[51] E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors*, vol. 23, p. 5941, 2023, doi: 10.3390/s23135941.

[52] G. Oikonomou et al., "The Contiki-NG open source operating system for next generation IoT devices," *SoftwareX*, vol. 18, 2022, doi: 10.1016/j.softx.2022.101089.

[53] Palo Alto Research Center, "Predicting Train Breakdowns with the Internet of Things – JR East Collaboration with PARC," 2017.

[54] J. Pan and J. Mcclannon, "Future edge cloud and edge computing for internet of things applications," *IEEE Internet of Things Journal*, 2018, doi: 10.1109/jiot.2017.2767608.

[55] D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "SciKit-Learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.

[56] I. Rish, "An Empirical Study of The Naive Bayes Classifier," in *IJCAI2001 workshop on empirical methods in artificial intelligence*, vol. 3, pp. 41-46, 2011.

[57] S. M. Sajjad et al., "Detection and blockchain-based collaborative mitigation of internet of things botnets," *Hindawi Limited*, 2022, doi: 10.1155/2022/1194899.

[58] M. Selvaraj and G. Uddin, "A large-scale study of IoT security weaknesses and vulnerabilities in the wild," 2023, doi: 10.48550/arXiv.2308.13141.

[59] H. Shah and A. K. Vyas, "A systematic review for 6G and beyond 6G enable IoT network," *IEEE*, 2024, doi: 10.1109/ic457434.2024.10486585.

[60] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy," in *International Carnahan Conference on Security Technology (ICCST)*, pp. 1-8, 2019, doi: 10.1109/CCST.2019.8888419.

[61] M. Shardlow, "An analysis of feature selection techniques," *The University of Manchester*, vol. 1, pp. 1-7, 2016.

[62] SonicWall, "2023 SonicWall Mid-Year Cyber Threat Report," 2023. [Online]. Available: https://www.sonicwall.com/2023-mid-year-cyber-threat-report

[63] G. Spathoulas, N. Giachoudis, G. Damiris, and G. Theodoridis, "Collaborative blockchain-based detection of distributed denial of service attacks based on internet of things botnets," *Future Internet*, vol. 11, no. 11, 2019, doi: 10.3390/fi11110226.

[64] D. Stefanescu, L. Montalvillo, P. Galan-Garcia, J. Unzilla, and A. Urbieta, "A systematic literature review of lightweight blockchain for IoT," *IEEE Access*, 2022, doi: 10.1109/access.2022.3224222.

[65] S. K. Tayyaba, M. A. Shah, O. A. Khan, and A. W. Ahmed, "Software defined network (SDN) based internet of things (IoT)," *ACM*, 2018, doi: 10.1145/3102304.3102319.

[66] B. Tekinerdogan, Ö. Koksal, and T. Çelik, "System Architecture Design of IoT-Based Smart Cities," *Applied Sciences*, vol. 13, no. 7, p. 4173, 2023, doi: 10.3390/app13074173.

[67] United States Cybersecurity and Infrastructure Security Agency, "Cybersecurity Best Practices for Smart Cities," 2023. [Online]. Available: https://www.cisa.gov/sites/default/files/2023-04/cybersecurity-best-practices-for-smartcities_508.pdf

[68] L. S. Vailshery, "Internet of Things (IoT) - statistics & facts," 2024. [Online]. Available: https://www.statista.com/topics/2637/internet-of-things/#topicOverview

[69] [69] A. Viloria, O. B. Pineda Lezama, and N. Mercado-Caruzo, "Unbalanced data processing using oversampling: Machine learning," *Elsevier BV*, 2020, doi: 10.1016/j.procs.2020.07.018.

[70] R. Vishwakarma and A. K. Jain, "A survey of DDoS attacking techniques and defence mechanisms in the IoT network," *Springer Science and Business Media LLC*, 2019, doi: 10.1007/s11235-019-00599-z.

[71] A. Zafar et al., "An advanced strategy for addressing heterogeneity in SDN-IoT networks for ensuring QoS," *MDPI AG*, 2023, doi: 10.3390/app13137856.

[72] Sfar, E. Natalizio, Y. Challal, and Z. Chtourou, "A Roadmap for Security Challenges in the Internet of Things," *Digital Communications and Networks*, vol. 4, no. 2, pp. 118-137, 2018.

[73] S. Alqahtani, M. Alhassan, and A. Alzahrani, "Machine Learning-Based Intrusion Detection for IoT Networks: A Comprehensive Review," *IEEE Access*, 2024.